Explaining Group Recommendations via Counterfactuals

Maria Stratigi[a,*], Nikos Bikakis[b,c] and Kostas Stefanidis[a]

[a]Data Science Research Centre, Tampere University, Tampere, Finland
[b]Hellenic Mediterranean University, Chania, Greece
[c]Athena Research Center, Greece

ARTICLE INFO

Keywords:
Group Recommendations
Explainability
Explainable Recommender Systems
Fairness in Recommendation

ABSTRACT

Group recommender systems help users make collective choices but often lack transparency, leaving group members uncertain about why items are suggested. Existing explanation methods focus on individuals, offering limited support for groups where multiple preferences interact. In this work, we introduce and formalize the problem of finding counterfactual explanations for group recommendations. We introduce a comprehensive set of item- and explanation-level metrics that capture different aspects, such as relevance, popularity, explanation utility, interpretability, and fairness. We design five algorithms that employ techniques such as search-space pruning, greedy selection, early stopping, and incremental refinement to reduce computational cost and the number of recommender calls while producing high-quality explanations. The algorithms consider different trade-offs; for example, some prioritize computational efficiency at the expense of explanation quality, while others focus on improving explanation quality at a higher computational cost. Experiments on real-world datasets analyze these trade-offs in terms of explanation size, computational cost, utility, and fairness.

1. Introduction

Recommender systems have become indispensable in a wide range of online platforms, from e-commerce and streaming services to tourism and education. While most research in recommender systems has traditionally focused on individuals, many real-world decision-making processes involve groups of users, e.g., families choosing a movie, friends selecting a restaurant, or colleagues planning a trip. Group recommender systems aim to support such scenarios by aggregating individual preferences into a joint recommendation. Despite their growing importance, they often remain opaque: users typically see the final output but have little insight into why certain items were recommended to the group. This lack of transparency undermines user trust and limits acceptance, particularly in settings where group members need to justify or negotiate decisions.

Explainability has emerged as a critical requirement for recommender systems [46]. Existing research on explainable AI has explored various forms of explanations, including feature-based justifications [28], rule-based reasoning [47], and, more recently, counterfactual explanations [40]. Counterfactuals answer "what if" questions by identifying the changes needed in the input that would alter a recommendation outcome. For instance, a system may explain the appearance of a movie in a user's list by stating: "*If you had not rated item X, this recommendation would not appear*". Counterfactual explanations are actionable, model-agnostic, and easily interpretable, making them suitable for increasing transparency in black-box systems. However, prior work has almost exclusively focused on individual users [12, 18, 19, 5], leaving group settings largely unexplored.

Explaining group recommendations introduces unique challenges. Unlike individuals, groups combine multiple preference profiles, often leading to complex dynamics of influence and compromise. A counterfactual explanation must therefore account not only for the system's internal logic but also for how group members' interactions collectively shape the outcome. Moreover, group explanations raise additional concerns of fairness: if an explanation suggests removing only one member's interactions, that user may perceive the system as biased or exclusionary. Balancing interpretability, utility, and fairness is thus essential for explanations to be both informative and acceptable in group contexts [45].

A *counterfactual explanation for a group recommendation* identifies which past interactions of the group members are responsible for the presence of a particular item (the *target item*) in the group's recommendation list. To find such an explanation, we consider a set of items with which the group members have previously interacted and simulate their removal from the group's interaction history. We then rerun the group recommender and examine whether the target item still appears in the resulting recommendation list. If the target item no longer appears, the removed items constitute a valid counterfactual explanation, as their removal is sufficient to change the recommendation outcome. The overall explanation process is outlined in Figure 1 (more details in Sec. 2). The problem can be illustrated through the following question and its corresponding explanation.

Question
"*Why does the target item t appear in the recommendation list?*"

Explanation
"*If you had not interacted with the items X, item t would not have appeared in the recommendation list.*"

*Corresponding author
maria.stratigi@tuni.fi (M. Stratigi); bikakis@hmu.gr (N. Bikakis); konstantinos.stefanidis@tuni.fi (K. Stefanidis)
ORCID(s): 0000-0003-2482-4605 (M. Stratigi); 0000-0001-6859-1941 (N. Bikakis); 0000-0003-1317-8062 (K. Stefanidis)

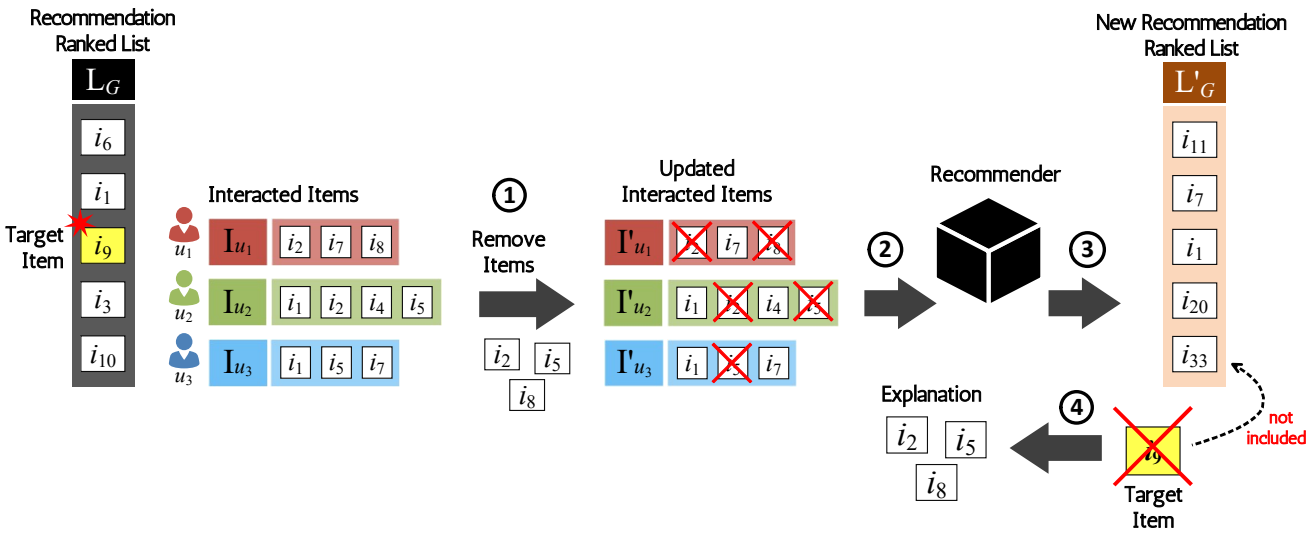


Figure 1: Explanation Process: Finding Counterfactual Explanations

One major advantage of our approach is that the explanations are formed from data items with which the group has previously interacted. This ensures that explanations remain grounded in the system's internal knowledge and do not rely on external information, thereby *preserving user trust*. Moreover, our approach is *model-agnostic and applicable to black-box recommender systems*, as it does not require access to their internal workings, but only to their outputs.

However, finding explanations is computationally challenging, particularly for groups. Since a group consists of multiple users, each with potentially numerous interactions, the set of candidate items to be examined is large. Furthermore, identifying an explanation requires examining all the different combinations of these items, whose number grows exponentially with respect to the number of items. As a result, exhaustive search becomes prohibitively costly, especially since evaluating each candidate subset requires invoking the group recommender.

To this end, we propose a framework comprising five efficient algorithms that employ different techniques, including search-space pruning, greedy selection, early stopping, backward pruning, and incremental refinement. These techniques aim to reduce the number of candidate explanations examined and the required recommender calls, while considering different trade-offs between computational efficiency and explanation quality. Moreover, we introduce a set of metrics that capture different dimensions of item relevance, including the extent of the group's interactions with an item, its system-wide popularity, its contribution to removing the target item from the recommendation list, and group members' preferences for the target item.

Specifically, we design five algorithms that aim to maximize explanation utility while minimizing computational cost. First, the *Pareto-filtering* algorithm exploits Pareto dominance to prune candidate items and reduce the search space, requiring a limited number of recommender calls. The *GreedyGrow* algorithm ranks candidate items based on their scores and incrementally adds them until a valid counterfactual explanation is found. The *ExpRebuild* and *Grow&Prune* algorithms extend *GreedyGrow* by introducing additional refinement steps. In particular, the *ExpRebuild* algorithm orders items according to their ability to explain the target item and then incrementally reconstructs the counterfactual. By contrast, *Grow&Prune* starts from an initial counterfactual and iteratively removes unnecessary items to produce a more concise explanation. Notably, explanation size is a crucial factor in explanation clarity [32, 23] (Sec. 4.1). Finally, *FixedWindow* applies a sliding window over the ranked candidate items to identify a valid counterfactual, and then performs an exhaustive search within the restricted window to find a more compact explanation.

Together, these algorithms explore different trade-offs among computational efficiency, explanation size, utility, and fairness, which we evaluate in our experiments.

Contribution. The main contributions are the following:

- **Problem Definition.** We introduce and formally define the problem of finding counterfactual explanations in group recommender systems.
- **Item and Explanation Metrics.** We introduce a comprehensive set of item-level metrics that assess recognition, likeness, expressivity, and influence, both within and beyond the group. Additionally, we define a set of explanation-level metrics that evaluate desirable properties, such as clarity, interpretability, cost, and fairness.
- **Algorithms.** We design five algorithms for efficiently discovering group counterfactual explanations, with different trade-offs regarding computational efficiency, cost of recommender calls, and explanation quality. We also provide a thorough complexity analysis of each algorithm in terms of computational time, the

number of recommender calls, and space requirements.

- **Experimental Evaluation.** We evaluate the proposed algorithms on two real-world datasets (MovieLens and Amazon). Our results highlight the trade-offs among the algorithms in terms of explanation quality, explanation size, computational cost, and fairness.

Previous Work. A preliminary version of this work appeared at the International Workshop on Design, Optimization, Languages and Analytical Processing of Big Data (DOLAP) 2025 [41]. The present paper substantially extends that study in three directions. (1) *We considerably extend the algorithmic framework* by refining the original Sliding Window method into FixedWindow, and introducing three additional algorithms (GreedyGrow, Grow&Prune, and ExpRebuild), and developing a Pareto-based filtering mechanism to reduce the search space. (2) *We introduce a richer set of item-level and explanation-level metrics*, including new measures of interpretability, utility, and fairness. (3) *We substantially expand the experimental evaluation* by replacing MovieLens 100K with MovieLens 10M and introducing an additional large-scale dataset, Amazon. Moreover, we provide a more comprehensive experimental analysis, examining several new factors and parameters, such as computational cost, explanation size, interpretability, utility, and fairness.

Outline. The remainder of this paper is organized as follows. Section 2 presents the basic concepts of the counterfactual explanation model. Then, Section 3 introduces the item-level metrics. Section 4 presents explanation-level metrics and defines the problem. Next, Section 5 presents and analyzes the proposed algorithms. Section 6 outlines the experimental setup and discusses the evaluation results. Section 7 reviews related work. Finally, Section 8 concludes the paper.

2. Group Counterfactual Explanations Model

To enhance the transparency and interpretability of group recommendation systems, we introduce a model for generating counterfactual explanations tailored to groups of users. These explanations provide actionable insights by identifying which items would lead to different recommendation outcomes if removed from the group's interaction history. In this section, we present the foundational concepts of our model, formalize the definition of group counterfactual explanations, and introduce relevant item-level metrics to support the explanation generation process. Table 1 summarizes the notation used throughout the paper.

2.1. Basic Concepts

We begin by defining the core entities involved in our model, namely users, groups, their interactions with items, and how these interactions shape group recommendations.

Table 1
Summary of Common Notations

Notation	Description
Users and Items	
U, u	Set of all users, a user
G	A group of users
I, i	Set of all items, an item
t	Target item, i.e., the item to be explained
I_u	Items with which user u has interacted
I_G	Items with which any member of G has interacted
$\rho_{i,u}$	Rating assigned by user u to item i
Recommendations	
$\pi(I)$	Group recommendation system given item set I
L_I	Group ranked recommendation list produced from I
$\mathsf{rank}(i, L_I)$	Rank position of item i in list L_I
Explanations and Metrics	
E	Counterfactual explanation
$\mathsf{rc}(i, S)$	Average recognition of item i among users S
$\mathsf{rt}(i, S)$	Average rating of item i by users S
$\mathsf{expwr}(P, t, I)$	Explanatory power of item subset P with respect to target item t and item set I
$\mathsf{infl}(i, t, I, G)$	Influence of item i on target item t for group G
$i.score$	Total score of item i
$\mathsf{fair}(G, E)$	Fairness of explanation E with respect to group G
$\mathsf{min}(E)$	Minimality of explanation E
$\mathsf{interpr}(E)$	Interpretability of explanation E
$\mathsf{cost}(E)$	Computational cost of explanation E
$\Omega(E)$	Utility of explanation E

User, Group & Interacted Items. Let U denote the set of all *users* in the system. For each user $u \in U$, let I_u represent the set of *items that the user has interacted with*, such as items they have ordered, rated, clicked, or liked. Let $G \subseteq U$ be a group of users, and let I_G denote the set of *items that any group member has interacted with*, formally defined as $I_G = \bigcup_{\forall u \in G} I_u$. Furthermore, $\rho_{i,u} \in [0, 1]$ represents the *rating assigned by user u* to *item i*. If a rating is not present, it is assumed to be zero. It is important to note that our approach does not require the presence of ratings for all users or items.

Recommendations. Let $\pi(I)$ denote a *group recommendation system*, which, given a set of interacted items I, produces an ordered *recommendation list* L_I, i.e., $\pi(I) = L_I$. For a given recommendation list L_I, let $\mathsf{rank}(i, L_I) \in [1, |L_I|]$ denote the *position (rank) of item i* in L_I. Generating a recommendation list L_I is associated with a *recommender call cost* ϕ, such as system latency or API usage cost.

In this work, we treat the group recommender system as a *black box*, enabling general applicability of our approach regardless of the internal logic or type of recommendation algorithm employed.

2.2. Group Counterfactual Explanations

Having defined the basic entities and the recommendation mechanism, we now formalize the concept of counterfactual explanations in group settings. Inspired by prior work on counterfactuals in individual recommendations [19], we extend the concept to group scenarios, where explanations must account for collective interactions rather than those of a single user.

The essence of a counterfactual explanation lies in identifying the set of previously interacted items that, if removed, would prevent the appearance of a specific item (the *target item*) in the recommendation list.

Group Counterfactual Explanations. Formally, a *group counterfactual explanation* is defined as follows. Given a group $G \subseteq U$ and a target item $t \in L_{I_G}$, a *group counterfactual explanation* $E \subset I_G$ is a set of interacted items such that if the group had not interacted with these items, the item t would not have been recommended[1].

Considering the above, the following holds. Let $\pi(I_G) = L_{I_G}$ with $t \in L_{I_G}$. Then, a set of items $E \subset I_G$ is a *counterfactual explanation* if $\pi(I_G \backslash E) = L_{I'_G}$ and $t \notin L_{I'_G}$.

Factual and Counterfactual Scenario. In this setting, the *factual scenario* is defined by the current interaction set I_G, where $\pi(I_G) = L_{I_G}$ and $t \in L_{I_G}$. The *counterfactual scenario* involves a modified interaction set $I'_G = I_G \backslash E$, such that $\pi(I'_G) = L_{I'_G}$ and $t \notin L_{I'_G}$. The difference $E = I_G \backslash I'_G$ thus constitutes the counterfactual explanation. This approach inherently assumes that the only *feasible action* is the *removal of items* from the group's interaction history.

Explanation Process Example. Figure 1 illustrates the overall explanation process. Suppose the recommendation list is L_G and the *target item* is i_9, i.e., the item for which we aim to generate an explanation. The group consists of *three users*: u_1, u_2, and u_3, each associated with their respective *interaction lists*, denoted as I_{u_1}, I_{u_2}, and I_{u_3}. ① Initially, we remove a selected set of items from the users' interaction lists, i.e., i_2, i_5, and i_8. This set represents a candidate counterfactual explanation. ② Next, we input the modified interaction lists I'_{u_1}, I'_{u_2}, and I'_{u_3} into the recommendation system. ③ The system returns a new group recommendation list L'_G. ④ Since the target item i_9 no longer appears in L'_G, the removed items i_2, i_5, and i_8 constitute a counterfactual explanation.

[1]For the remainder of this paper, *Counterfactual explanation* will be referred to interchangeably as *Counterfactual* or *Explanation*.

3. Item Metrics

To support the generation and evaluation of counterfactual explanations, we define a set of item-level metrics that quantify various aspects of item influence and group interaction patterns.

Item Recognition. We begin by defining how widely the users know or recognize an item. The *item recognition* metric captures the fraction of users in a given set who have interacted with the item.

Let i be an item and $S \subseteq U$ a set of users. Then, item recognition $\mathsf{rc}(i, S) \in [0, 1]$ is defined as:

$$\mathsf{rc}(i, S) = \frac{\sum_{\forall u \in S} \zeta(u, i)}{|S|} \tag{1}$$

where $\zeta(u, i) = 1$ if user u has interacted with item i (i.e., $i \in I_u$), and 0 otherwise.

We evaluate this item recognition in two contexts: (1) $\mathsf{rc}(i, G)$: among the group members (*group item recognition*); and (2) $\mathsf{rc}(i, U \backslash G)$: among users outside the group *(public item recognition)*.

Item Rating. Next, we consider the average quality assessment of an item based on user feedback. Given an item i and a user set $S \subseteq U$, the *item rating* is the average of ratings provided by S users:

$$\mathsf{rt}(i, S) = \frac{\sum_{\forall u \in S} \rho_{i,u}}{|S|} \tag{2}$$

Similar to recognition, item rating is computed in two contexts: (1) $\mathsf{rt}(i, G)$: among the group members *(group item rating)*; and (2) $\mathsf{rt}(i, U \backslash G)$: among all users excluding the group (*public item rating*).

Item Influence on the Target Item. The *influence of an item on the target item* quantifies the extent to which a previously interacted item contributes to the recommendation score assigned to the target item t by the recommender system, in a *model-agnostic* setting.

Intuitively, interacted items that lead to a higher recommendation score for t, resulting in t being ranked higher, are considered to exert a stronger influence. These items are therefore more likely to appear in counterfactual explanations, as their removal can significantly alter the ranking of t.

To estimate this influence in a model-agnostic way, we treat each user in the group individually and evaluate the effect of their interactions on the recommendation score of t. Specifically, for each user $u \in G$, we call the group recommender using only their interaction history I_u and retrieve the recommendation score for t.

Formally, let $G' = \{u \in G \mid i \in I_u\}$ be the set of group members who have interacted with item i. Then, the *influence* $\mathsf{infl}(i, t, I, G) \in [0, 1]$ of item i on target item t for

group G is computed as the average recommendation score of t across G' users:

$$\mathsf{infl}(i, t, I, G) = \sum_{\substack{\forall u \in G' \\ I_u \subseteq I}} \mathsf{recScore}(t, I_u) \Big/ |G'| \tag{3}$$

Here, $\mathsf{recScore}(t, I_u) \in [0, 1]$ denotes the recommendation score assigned to t by the recommender system when invoked with only user u interacted items I_u. This approach allows us to estimate item influence without requiring access to the internal workings of the recommender (i.e., in a *model-agnostic* way).

Item Set Explanatory Power. We introduce the *explanatory power* metric, which quantifies the potential of an item set to contribute to a counterfactual explanation and, consequently, its ability to explain the presence of a target item. The metric is computed in a *model-agnostic* manner, relying solely on observable changes in the recommendation output.

Let I be the interacted items set, L_I the resulting recommendation list with $|L_I| = m$, and t the target item. Let $P \subset I$ be a subset of items considered for removal, and let $\pi(I \backslash P)$ be the new recommendation list.

The *explanatory power* of item set P, denoted $\mathsf{expwr}(P, t, I) \in [0, 1]$, is defined as:

$$\mathsf{expwr}(P, t, I) = \begin{cases} 1, & \text{if } t \notin \pi(I \backslash P), \\ \frac{\mathsf{rank}(t, \pi(I \backslash P)) - 1}{m}, & \text{otherwise.} \end{cases} \tag{4}$$

Explanatory power quantifies the rank of t when the item set P is removed from the interacted item set I. Higher values indicate that the items in P are more likely to be part of an explanation.

Particularly, if t is eliminated from the list ($t \notin \pi(I \backslash P)$), then P forms a counterfactual and its explanatory power is set to one. On the other hand, if P results in item t being ranked top (i.e., top-1), explanatory power takes its lowest value of zero.

Item Total Score. While individual metrics (e.g., recognition, rating, influence) offer valuable insights, combining them into a single score enables more effective ranking and selection of items for explanation. To this end, we define the *item total score* as an aggregate of multiple item-based metrics.

Given an item i and a collection of k metrics $(f_1, f_2, \ldots, f_k)$, we compute the total score as:

$$i.score = \sum_{j=1}^{k} f_j(i) \tag{5}$$

In this work, we use the following five metrics to calculate the total score: (1) $\mathsf{rc}(i, G)$: group item recognition; (2) $\mathsf{rc}(i, U \backslash G)$: public item recognition; (3) $\mathsf{rt}(i, G)$: group item rating; (4) $\mathsf{rt}(i, U \backslash G)$: public item rating; and (5) $\mathsf{infl}(i, t, I, G)$: item influence on the target item.

Putting it all together, the final formula for the *item total score* is:

$$\begin{aligned} i.score = \mathsf{rc}(i, G) + \mathsf{rc}(i, U \backslash G) + \mathsf{rt}(i, G) \\ + \mathsf{rt}(i, U \backslash G) + \mathsf{infl}(i, t, I, G) \end{aligned} \tag{6}$$

This score serves as a unified measure of an item's importance, considering both its relevance within the group and its potential impact on the target recommendation outcome. It plays a central role in guiding the selection of "explanatory" items in our counterfactual framework.

4. Counterfactual Metrics & Problem Definition

In this section, we present a set of evaluation dimensions for assessing the quality of counterfactual explanations in group recommender systems. Although prior research on counterfactual reasoning has largely centered on individual-level explanations, extending these approaches to group settings presents unique challenges and design considerations. Beyond generating explanations that are minimal and interpretable, we must also consider the computational cost of discovering them, the overall utility they offer to the group, and the fairness of how explanatory changes are distributed among group members.

We begin by formalizing utility-oriented metrics that capture properties such as explanation minimality, interpretability, and generation cost, and introduce a unified utility function that balances these aspects. We then define a notion of counterfactual fairness tailored to groups, ensuring that no single user is disproportionately affected by the changes recommended in the explanation. Collectively, these metrics enable a comprehensive assessment of group-level counterfactual explanations, emphasizing clarity, utility, and fairness.

4.1. Counterfactual Metrics

In this section, we introduce a set of metrics to assess the quality and characteristics of group-level counterfactual explanations in recommender systems. These metrics capture different desirable properties of explanations, including clarity, interpretability, cost-efficiency, and fairness.

Counterfactual Explanation Minimality. We use the notion of counterfactual *minimality* (also referred to as *sparsity*) to evaluate the *clarity* of an explanation; i.e., how easily it can be understood. Numerous studies in cognitive psychology and explainable AI have shown that explanations involving fewer changes are more understandable and cognitively efficient to process [32, 23, 36, 31, 50, 12, 49].

In our setting, the minimality of a counterfactual explanation E is quantified by the number of items it contains. A smaller $|E|$ indicates a clearer explanation. This quantity corresponds to the number of user-item interactions removed to achieve the counterfactual.

We formally define the *counterfactual explanation minimality* as:

$$\mathsf{min}(E) = 1 - \frac{|E|}{|I_G|} \tag{7}$$

where $|I_G|$ is the total number of interacted items by the group. The value is normalized in $[0, 1]$ such that higher values indicate better minimality.

Counterfactual Explanation Interpretability. An explanation is more interpretable when the items it involves are familiar to users [9]. We measure interpretability through item recognition, i.e., how often an item appears in user interaction histories. This includes both (1) the recognition of an item within the group, and (2) its recognition by the broader user base.

To quantify this, we define the interpretability of a counterfactual explanation E as the average recognition of its items:

$$\mathsf{interpr}(E) = \frac{1}{2|E|}\Big(\sum_{\forall i \in E} \mathsf{rc}(i, G) + \sum_{\forall i \in E} \mathsf{rc}(i, U \setminus G)\Big) \tag{8}$$

Here, $\mathsf{rc}(i, G)$ denotes the recognition of item i in the group, and $\mathsf{rc}(i, U \setminus G)$ represents its recognition among other users.

Counterfactual Explanation Cost. Producing a counterfactual explanation involves making multiple calls to the recommender system, each associated with a cost ϕ. The total cost of an explanation E is thus defined as the number of calls made to discover it:

$$\mathsf{cost}(E) = \sum_{i=1}^{n} \phi \tag{9}$$

where n is the number of recommender invocations required. For simplicity, we assume $\phi = 1$ in our evaluation, which allows us to count the number of calls directly.

Counterfactual Explanation Utility. We define the *utility* of a counterfactual explanation E as a combination of its clarity and interpretability. The utility function is defined as:

$$\Omega(E) = \psi\big(\mathsf{min}(E), \mathsf{interpr}(E)\big) \tag{10}$$

where ψ is an aggregation function that balances the two metrics, for instance via a weighted sum or product.

4.2. Problem Definition

We formally define the problem of finding *Group Counterfactual Explanation* as follows.

Group Counterfactual Explanation Problem. Given a *group* G; a *group interacted items* I_G; a *group recommended items list* L_{I_G}; a *target item* t; a *recommender call cost* ϕ; and a *budget* B in terms of recommender call cost; our goal is to find a *group counterfactual explanation* E^*, such that the *explanation utility* $\Omega(E^*)$ is maximized and the *counterfactual cost* $\mathsf{cost}(E^*)$ does not exceed the budget B.

$$E^* = \arg\max \Omega(E) \quad \text{s.t.} \quad \mathsf{cost}(E) \leq B$$

Computational Complexity. Solving this problem involves evaluating all subsets of I_G, i.e., the power set excluding the empty set. Therefore, the number of candidates is $2^{|I_G|} - 1$. Since each candidate requires a recommender system call with cost ϕ, the computational complexity is $O(\phi \cdot 2^{|I_G|})$.

4.3. Counterfactual Fairness

The fairness notion used in this work is inspired by *individual fairness* [11, 7, 26], which states that similar individuals should be treated similarly. We translate this to the group setting by requiring that the burden of a counterfactual explanation (i.e., the items removed) be fairly distributed among group members.

Formally, we say that an explanation E is fair if each group member contributes a similar number of interacted items to the explanation. For any user $u \in G$, let $\zeta(u, i)$ be an indicator function equal to 1 if user u has interacted with item i, and 0 otherwise.

To assess fairness, we compute the standard deviation of the number of items from each user's profile that are included in E:

$$\mathsf{fair}(G, E) = \frac{1}{\xi + \sigma\left(s_{u_1}, s_{u_2}, \ldots, s_{u_{|G|}}\right)} \tag{11}$$

$$s_u = \sum_{\forall i \in E} \zeta(u, i), \qquad \text{where } u \in G.$$

A lower standard deviation indicates more balanced participation and, therefore, greater fairness. By taking its inverse, we ensure that higher values of $\mathsf{fair}(G, E)$ correspond to fairer explanations. A small constant $\xi > 0$ is added to avoid division by zero. Perfect fairness is achieved when all users contribute equally, i.e., $\sigma = 0$.

Note that fairness is evaluated as an ex-post property of the generated explanations. The proposed algorithms do not explicitly optimize fairness; rather, we investigate how their different selection and refinement strategies affect the distribution of explanatory contributions among group members.

5. Algorithmic Framework for Counterfactual Search

The search space for group counterfactual explanations is exponential, making exhaustive search impractical. We therefore develop efficient algorithms that use several techniques to select and refine candidate explanations. In the following sections we present five algorithms. We first introduce *ParetoFiltering* (Sec. 5.1), which selects items based

on multidimensional dominance and can serve as a preprocessing step, which prunes the candidate item space early, requiring only a limited number of recommender calls. *FixedWindow* (Sec. 5.2) scans a sorted candidate list using a sliding window to locate a counterfactual, within which an exhaustive local search is performed to extract a more compact explanation. *GreedyGrow* (Sec. 5.3) follows a forward counterfactual construction strategy, progressively adding items until a counterfactual is found. The following two methods refine this procedure. *Grow&Prune* (Sec. 5.4), which starts from an initial counterfactual and iteratively eliminates items to reduce the explanation size; and *ExpRebuild* (Sec. 5.5), which ranks items by their ability to explain the target item and incrementally reconstructs the counterfactual.

5.1. Search Space Reduction Using Pareto-based Filtering

We first introduce **ParetoFiltering**, which uses Pareto dominance across item metrics to select candidate items. The key goal of this method is to improve efficiency by pruning dominated items before counterfactual search, thereby reducing the number of recommender calls required. Hence, Pareto filtering can be considered as a preprocessing step to effectively reduce the search space.

Specifically, we leverage the item metrics introduced in Section 3 to map each item into a multidimensional metric space, enabling dominance-based filtering to identify a "high-quality" counterfactual superset.

Metric-based Representation. Let d denote the number of item-level metrics. Each item $i_k \in I_G$ is represented as a d-dimensional vector $o_k = (o_k^1, o_k^2, \ldots, o_k^d)$, where $o_k^j = f_j(i_k)$ is the value of metric j for item i_k. The attribute space is defined over $\mathbb{R}^d$.

Pareto Set. An object o_1 *dominates* o_2, denoted $o_1 \succ o_2$, if: (1) $\forall w \in [1,d],\ o_1^w \geq o_2^w$, and (2) $\exists j \in [1,d],\ o_1^j > o_2^j$. The *Pareto set*, or skyline [24], denoted $PS(O)$ for object set O, is defined as:

$$PS(O) = \{o_i \in O \mid \nexists o_k \in O : o_k \succ o_i\}.$$

Items in $PS(O)$ are referred to as Pareto-optimal items and represent a non-dominated frontier within the metric space.

Threshold-based Generalization. To flexibly control the cardinality of the result set, we generalize the Pareto set using a threshold vector $\tau = \langle \tau^1, \tau^2, \ldots, \tau^d \rangle$ where $\tau^j \in \mathbb{R}$ adjusts strictness per metric.
An object o_1 *τ-dominates* o_2, written $o_1 \succ_\tau o_2$, if: (1) $\forall w \in [1,d],\ o_1^w \geq o_2^w + \tau^w$, and (2) $\exists j \in [1,d],\ o_1^j > o_2^j$. The *$\tau$-Pareto set* is then defined as:

$$PS_\tau(O) = \{o_i \in O \mid \nexists o_k \in O : o_k \succ_\tau o_i\}.$$

Setting $\tau = 0$ yields the standard Pareto set; $\tau < 0$ relaxes dominance criteria (reducing the size of $PS_\tau(O)$ set) while $\tau > 0$ tightens them (increasing the size of $PS_\tau(O)$ set). Efficient skyline algorithms like D&C [24], SFS [10], LESS [13], and SaLSa [6] can be adapted to compute $PS_\tau(O)$.

Pareto-based Filtering Technique Overview. To efficiently (i.e., using a small number of recommender calls) identify a counterfactual explanation set, we adopt an iterative filtering strategy based on τ-Pareto dominance. Specifically, we remove from the group's interaction items all items belonging to the τ-Pareto set, which represent a collection of "high-impact" items across the item-level metrics. The group recommender is then invoked to assess whether excluding these items causes the target item to disappear from the recommendation list. In such a case, the τ-Pareto set is selected as the candidate counterfactual explanation.

In cases where the target item persists in the recommendation list, the examined τ-Pareto set is not a counterfactual. To explore a broader set of items, we incrementally relax the dominance criteria by increasing the threshold vector τ, and recompute the τ-Pareto set. This threshold-based relaxation enables a controlled expansion of the candidate item space. The process is repeated until a τ-Pareto set corresponds to a counterfactual explanation.

Algorithm Description. Algorithm 1 takes as input an interacted item set I and a target item t, and returns a counterfactual explanation CS. Items are first mapped to d-dimensional metric vectors based on items metrics values (*lines* 2–4). The main loop (*lines* 7–14) iteratively relaxes the Pareto dominance criteria. At each iteration, a threshold vector τ is computed based on the standard deviation of each metric dimension (*line* 9), and the τ-Pareto set is computed (*line* 10). The items that correspond to the τ-Pareto set objects are inserted into the item set PI (*line* 11). The procedure isCF (Proc. 1) is then invoked to determine if PI corresponds to a counterfactual explanation (*line* 12). Threshold relaxation continues until a counterfactual explanation is found.

Complexity: Time, Space, and Recommender Calls.
Time Complexity: In Algorithm 1, *line* 3, considering the item metrics used in our implementation, the worst-case computational cost for computing the *total item score* (Eq. 6) is analyzed as follows.

The cost of computing the two item *recognition metrics*, $\mathsf{rc}(i,G)$ and $\mathsf{rc}(i,U\backslash G)$ (Eq. 1), is $O(max(|U|,|I|))$ per item (if hash-based indexes are used). For simplicity, let us assume that $|U| > |I|$, so the complexity is $O(|U|)$. The same holds for the *rating metrics*, $\mathsf{rt}(i,G)$ and $\mathsf{rt}(i,U\backslash G)$ (Eq. 2); both require $O(|U|)$ per item. In order to compute the *influence metric*, $\mathsf{infl}(i,t,I,G)$ (Eq. 3), for each user in G, we need to invoke the recommender. Hence, the cost of computing influence metric is $O(|U|\phi)$, where ϕ is the recommender invocation cost.

Therefore, the total cost for computing all metrics for a single item is $O(|U|\phi)$, and for all items I is: $O(|I||U|\phi)$.

Algorithm 1. ParetoFiltering (I, t)

Input: I: interacted items; t: target item
Output: CS: counterfactual explanation;
Parameters: π: group recommender;
$f_1, f_2, \ldots, f_d$: item metrics functions
Variables: i: item; O: d-dimensional objects;
it: iteration counter; τ: attributes thresholds;
PS: τ-pareto set; PI: items of τ-pareto set;
cfFound: boolean variable

1 $O \leftarrow \varnothing$
2 **foreach** $i \in I$ **do** //transform items to d-dimensional objects
3 $o_i \leftarrow (f_1(i), f_2(i), \ldots, f_d(i))$ //compute the d metrics for item i
4 $O \leftarrow O \cup o_i$
5 $PI \leftarrow \varnothing$
6 $it \leftarrow 0$
7 **do**
8 **if** $PI = I$ **then break** // all items examined – counterfactual not exist
9 $\tau \leftarrow \left\langle it \cdot \sigma(\bigcup_{\forall o_i \in O} o_i^1), it \cdot \sigma(\bigcup_{\forall o_i \in O} o_i^2), \ldots, it \cdot \sigma(\bigcup_{\forall o_i \in O} o_i^d) \right\rangle$
//compute attributes' thresholds values as attributes std – assign to τ the negative values
10 $PS \leftarrow$ find τ-pareto set for objects O
11 $PI \leftarrow$ get the items of I that correspond to τ-pareto set PS objects //items set to be checked as counterfactual
12 $cfFound \leftarrow \mathsf{isCF}(I, PI, t)$ //call recommender system
13 $it \leftarrow it + 1$
14 **while** ($cfFound = \mathsf{false}$)
15 **if** $cfFound = \mathsf{true}$ **then**
16 $CS \leftarrow PI$
17 **else** //counterfactual not exist
18 $CS \leftarrow \varnothing$
19 **return** CS

Procedure 1: isCF(I, S, t)

Input: I: interacted items; S: interacted items to be checked as counterfactual; t: target item;
Output: boolean value:(if S is counterfactual)
Parameters: π: group recommender
Variables: L: recommendation list generated considering $I \backslash S$ as interacted

1 $L \leftarrow \pi(I \backslash S)$ //call recommender system
2 **if** $t \notin L$ **then**
3 **return** true
4 **else**
5 **return** false

In the *second loop* (*line* 7), the standard deviation computation in *line* 9 costs $O(d|I|)$. In *line* 10 the τ-pareto set computation has a worst-case complexity of $O(|I| \log^{d-2} |I|)$ using partition-based algorithms [24], or $O(d|I|^2)$ for practical algorithms such as SFS [10], LESS [13], or SaLSa [6].

The procedure isCF in *line* 13 incurs a cost of $O(\phi)$. The loop (*line* 7) is executed up to a small constant ν times, as the thresholds τ quickly converge to include all items in I. Therefore, the second loop (*line* 7) has a worst-case complexity of: $O\left(\nu \cdot (d|I| + |I| \log^{d-2} |I| + \phi)\right)$.

Combining both loops, the overall *worst-case time complexity* of the *Pareto-filtering* algorithm is:

$$O\left(|I||U|\phi + \nu \cdot (|I| \log^{d-2} |I| + \phi)\right)$$

Space Complexity: The *space complexity* of the algorithm (excluding input data) is $O(|I|)$, accounting for internal structures O, PS, PI, and CS.

Recommender Calls: The *number of recommender calls* in the worst case is $O(|I||U| + \nu)$.

5.2. FixedWindow Search

The **FixedWindow** search technique serves as a local search strategy that incrementally examines contiguous subsets of items. The algorithm operates on a ranked item list and iteratively explores fixed-size windows, followed by an internal refinement phase to extract a small subset that corresponds to a counterfactual.

The interacted items are sorted by their total score, and a sliding window of fixed size is applied to the ordered list. The window starts at the beginning of the list and moves one position at a time, covering consecutive items in each step. At every position, the algorithm simulates the removal of the window items from the group's interaction items and then re-invokes the group recommender system. If the target item is no longer recommended to the group, the current window corresponds to an explanation. If the window traverses the entire list without yielding a counterfactual explanation, the window size is increased, and the list is scanned again from the beginning.

When the items within a window form a counterfactual explanation, we apply a *refinement phase* to improve minimality (i.e., reduce the explanation size). Specifically, we examine all possible subsets (i.e., the powerset) of the window items in ascending order of cardinality. The first subset that corresponds to a counterfactual is selected as the final explanation, ensuring conciseness and interpretability. Note that, since this refinement phase has exponential complexity, we keep the window size relatively small to balance explanation quality with computational efficiency.

Algorithm Description. Algorithm 2 receives as input an interacted items set I, a target item t, and an initial window size w, and returns a counterfactual explanation CF. Initially, the total item score is computed for each item in I (*lines* 2–4), and the items are sorted in descending order to form the ranked list H (*line* 5). Then, the current window size w_{cur} is initialized to the predefined input value w (*line* 6).

For each window size, the algorithm slides a contiguous window W over H (*line 9*) and checks whether the items W

Algorithm 2. FixedWindow(I, t, w)

Input: I: interacted items; t: target item; w: initial window size
Output: CF: counterfactual explanation
Parameters: π: group recommender; $f_1, f_2, \ldots, f_d$: item metric functions
Variables: H: ranked item list; W: current window; S: subset of window; w_{cur}: current window size

1 $H \leftarrow \emptyset$
2 **foreach** $i \in I$ **do** //compute items scores & initialize items list
3 $\quad i.score \leftarrow \sum_{g=1}^{d} f_g(i)$ //compute item score by aggregating item metrics
4 $\quad$ insert i into H
5 sort H in descending order based on item scores
6 $w_{cur} \leftarrow \min(w, |H|)$ //start with initial window size
7 **while** $w_{cur} \leq |H|$ **do**
8 $\quad$ **for** $i \leftarrow 1$ **to** $|H| - w_{cur} + 1$ **do** //sliding window search
9 $\quad\quad W \leftarrow H[i : i + w_{cur} - 1]$ //window of w_{cur} items
10 $\quad\quad$ **if** isCF(I, W, t) **then** //check if window is a counterfactual
11 $\quad\quad\quad$ **foreach** $S \in$ PowerSet(W) *in order of increasing* $|S|$ **do** //**refinement phase**: check subsets
12 $\quad\quad\quad\quad$ **if** isCF(I, S, t) **then**
13 $\quad\quad\quad\quad\quad CF \leftarrow S$
14 $\quad\quad\quad\quad\quad$ **break**
15 $\quad\quad\quad$ **if** $CF \neq \emptyset$ **then return** CF
16 $\quad$ **if** $w_{cur} = |H|$ **then return** $\emptyset$
17 $\quad w_{cur} \leftarrow \min(w_{cur} + w, |H|)$ //increase window size by initial w
18 **return** $\emptyset$

correspond to a counterfactual explanation by invoking the isCF procedure (which invokes the recommender) (*line* 10).

When a window contains a counterfactual explanation, the algorithm enters the refinement phase (*lines* 11–14), where all non-empty subsets of W are examined in increasing order of cardinality. The first subset S that corresponds to a counterfactual is returned (*line* 12–15).

If no counterfactual window is found, the window size is increased by w and the search is repeated (*line* 17). The algorithm terminates when either an explanation is identified (*line* 15) or all window sizes are examined (i.e., $w_{cur} = |H|$).

Complexity: Time, Space, and Recommender Calls.
Time Complexity: In Algorithm 2, *line* 3, each item $i \in I$ is assigned a score by aggregating the values of d metric functions $f_1, f_2, \ldots, f_d$. As shown in Section 5.1 the complexity to compute the metric functions for an item is $O(|U|\phi)$. Therefore, the cost of computing all scores is: $O(|I||U|\phi)$.

Sorting the ranked list H (*line* 5) employing a linearithmic sorting algorithm (e.g., mergesort) costs $O(|I| \log |I|)$. In the main loop (*lines* 7–17) the number of distinct window sizes considered in the worst case is $\lceil |I|/w \rceil$. For each window size w_{cur}, the algorithm checks $O(|I| - w_{cur} + 1)$ windows. Hence, the total number of windows examined is $\sum_{k=1}^{\lceil \frac{|I|}{w} \rceil} (|I| - \min(kw, |I|) + 1)$, which corresponds to: $O\left(\frac{|I|^2}{w}\right)$.

For each window, the algorithm invokes the isCF procedure once (*line* 10), incurring a cost of $O(\phi)$. In *line* 11, the worst case, all $2^{w_{cur}} - 1$ subsets are evaluated, each requiring a call to isCF with cost $O(\phi + m)$ (*line* 12). Since w_{cur} can grow up to $|I|$, the refinement phase may require up to $2^{|I|}$ checks in the worst case. Thus, the computational cost of the main loop (*lines* 7–17) is:

$$O\left(\frac{|I|^2}{w}\phi + 2^{|I|}\phi\right)$$

Combining all parts, the overall *worst-case time complexity* of the *FixedWindow* algorithm is:

$$O\left(|I||U|\phi + |I| \log |I| + \frac{|I|^2}{w}\phi + 2^{|I|}\phi\right)$$

Space Complexity: The *space complexity* is $O(|I|)$, accounting for the sorted list H and the temporary subset storage during refinement.

Recommender Calls: The *number of recommender calls* in the worst case is:

$$O\left(|I||U| + \frac{|I|^2}{w} + 2^{|I|}\right)$$

Note that, in practice, the exponential term is rarely reached due to computational cut-offs, and w is typically chosen as a small constant, which keeps the quadratic term as the dominant factor.

5.3. GreedyGrow Search

The **GreedyGrow** search is a method that progressively expands the search window until a counterfactual explanation is found. Unlike the fixed-size sliding window approach, this technique selects the top-ranked items in the list and adds one item at a time in sequential order. At each iteration, the set of top-ranked items is checked to see if it corresponds to a counterfactual explanation.

This method is particularly effective at rapidly identifying explanations with strong influence; however, it has a notable limitation: the resulting explanations are not guaranteed to be minimal. Specifically, although the approach identifies a subset that successfully alters the recommendation outcome, it may include extraneous items that are not necessary for forming a counterfactual. Consequently, the final explanation may be "unnecessarily" large, which, as discussed in Section 4, can reduce interpretability and increase cognitive load for end users.

Algorithm Description. Algorithm 3 implements the *GreedyGrow* search strategy, takes as input the interacted item set I and a target item t, and returns the explanation

Algorithm 3. GreedyGrow (I, t)

Input: I: interacted items; t: target item;
Output: CF: counterfactual explanation;
Parameters: π: group recommender;
$f_1, f_2, \ldots, f_d$: item metrics functions;
Variables: H: items list; i: item; S: selected items set;
cfFound: boolean variable;

```
1  H ← ∅
2  foreach i ∈ I do                          //compute items scores & initialize items list
3      i.score ← Σ_{g=1}^{d} f_g(i)          //compute item score by aggregating item metrics
4      insert i into H
5  sort H in descending order based on item scores
6  S ← ∅
7  for j ← 1 to |H| do                       //select top items
8      S ← S ∪ H[j]                          //items set to be checked as counterfactual
9      cfFound ← isCF(I, S, t)               //call recommender system
10     if cfFound = true then break
11 if cfFound = true then
12     CF ← S
13 else                                      //no counterfactual exists
14     CF ← ∅
15 return CF
```

CF. The algorithm first computes the item total score for each interacted item and stores the items in a ranked list H (*lines* 2–5). It then initializes an empty candidate set S (*line* 6).

During each iteration of the main loop (*lines* 7–10), the next highest-ranked item from H is added to S, and the algorithm checks whether S constitutes a counterfactual explanation by invoking isCF (Procedure 1). The process terminates as soon as the target item is removed from the recommendation list.

If a counterfactual explanation is found, the current set S is returned as the final explanation (*line* 12). Otherwise, if all items are exhausted without success, the algorithm returns the empty set (*line* 14).

Complexity: Time, Space, and Recommender Calls.
Time Complexity: In *line* 3, each item $i \in I$ is assigned a score by aggregating the values of d metric functions $f_1, f_2, \ldots, f_d$. As shown in Section 5.1, the complexity to compute the metric functions for an item is $O(|U|\phi)$. Thus, computing all item scores has a total cost: $O(|I||U|\phi)$.

Sorting the scored list H (*line* 6) takes $O(|I| \log |I|)$. Then, in the main loop (*lines* 7–10), items from H are incrementally added to the set S. After each addition, the isCF function is called (*line* 9) to check whether S constitutes a valid counterfactual explanation. In the worst case, this process continues until all $|I|$ items have been added. Each call to isCF has cost $O(\phi)$. Thus, in the worst case, the total cost of the calls to isCF is: $O(|I|\phi)$.

From the above analysis, the overall *worst-case time complexity* of the *GreedyGrow* algorithm is:

$$O(|I||U|\phi + |I| \log |I|).$$

Space Complexity: The *space complexity* is $O(|I|)$, due to the storage of the list H and the growing set S.

Recommender Calls: The *number of recommender calls* in the worst case is $O(|I||U| + |I|) = O(|I||U|)$.

5.4. Grow&Prune Search

To reduce the size of explanations produced by GreedyGrow, we introduce the **Grow&Prune** search method, which first identifies an initial explanation and then refines it by improving minimality through backward elimination.

The *Grow&Prune* approach initially applies the *GreedyGrow* technique to generate an initial counterfactual explanation. Once this initial explanation is identified, we enter a *reduction phase that evaluates the necessity of each item within the explanation*. Each item in the initial counterfactual explanation is examined individually by temporarily excluding it from the set. For each exclusion, the group recommender is evaluated on the modified explanation to verify whether it continues to correspond to a counterfactual. If the exclusion results in a non-counterfactual explanation, the item is classified as essential and reinserted; otherwise, the exclusion is preserved, yielding a more concise explanation.

Algorithm Description. Algorithm 4 implements the *Grow&Prune* strategy. The algorithm first executes a grow phase (*lines* 1–10) identical to the *GreedyGrow* algorithm, incrementally adding items until a counterfactual explanation is identified. If no such explanation is found, the algorithm terminates and returns the empty set (*line* 20).

Once a counterfactual explanation is obtained, the items in the candidate set S are sorted in ascending order of score (*line* 12), and the algorithm enters the prune phase. In this phase, each item $i \in S$ is temporarily removed from the explanation (*line* 15), and the algorithm checks whether the reduced set still constitutes a valid counterfactual by invoking isCF (Proc. 1) (*line* 16).

If the counterfactual condition is preserved, the removal is retained (*line* 17); otherwise, the item is restored. The single-pass procedure continues until all items in the initial explanation have been examined once. The resulting set is returned as the final counterfactual explanation CF (*line* 18).

Complexity: Time, Space, and Recommender Calls.
Time Complexity: In *line* 3, each item $i \in I$ is assigned a score via the aggregation of d metric functions $f_1, \ldots, f_d$. As shown in Section 5.1, the complexity to compute the metric functions for an item is $O(|U|\phi)$. Thus, computing all item scores has a total cost: $O(|I||U|\phi)$.

Sorting the scored list H (*line* 6) takes $O(|I| \log |I|)$. Then, during the *grow phase* (*lines* 7–10), the algorithm incrementally builds a candidate set S by scanning items

Algorithm 4. Grow&Prune (I, t)

Input: I: interacted items; t: target item
Output: CF: counterfactual explanation
Parameters: π: group recommender;
$f_1, f_2, \ldots, f_d$: item metrics functions
Variables: H: items list; i: item; S: selected items set;
cfFound: boolean variable

1 $H \leftarrow \emptyset$
2 **foreach** $i \in I$ **do** //compute items scores & initialize items list
3 $i.score \leftarrow \sum_{g=1}^{d} f_g(i)$ //compute item score by aggregating item metrics
4 insert i into H
5 sort H in descending order based on items scores
6 $S \leftarrow \emptyset$
7 **for** $j \leftarrow 1$ **to** $|H|$ **do** //grow phase
8 $S \leftarrow S \cup H[j]$
9 *cfFound* $\leftarrow$ isCF(I, S, t)
10 **if** *cfFound* = true **then break**
11 **if** *cfFound* = true **then**
12 sort S in ascending order based on item scores
13 $CF \leftarrow S$
14 **foreach** $i \in S$ **do** //prune phase
15 $CF' \leftarrow CF \setminus \{i\}$
16 **if** isCF(I, CF', t) **then** //check if CF' still a counterfactual
17 $CF \leftarrow CF'$
18 **return** CF
19 **else** //counterfactual not found
20 **return** $\emptyset$

in H in descending order. After each addition, the isCF function is invoked. In the worst case, this loop iterates $|I|$ times, leading to: $O(|I| \cdot \phi)$.

In *prune phase* (*lines* 11–20), the items in S (of size at most $|I|$) are sorted in descending score order, costing at most $O(|I| \log |I|)$. Then, for each item $i \in S$, the algorithm checks whether removing i still yields a valid counterfactual by calling isCF. In the worst case, this requires $O(|I|)$ additional calls, resulting in: $O(|I| \log |I| + |I|\phi)$.

Combining all parts, the overall *worst-case time complexity* of the *Grow&Prune* algorithm is:

$$O(|I||U|\phi + |I| \log |I| + |I|\phi + |I| \log |I| + |I|\phi) = O(|I||U|\phi + |I| \log |I|)$$

Space Complexity: The *space complexity* is $O(|I|)$, due to the storage of lists H, S, and temporary counterfactual sets.

Recommender Calls: The *total number of recommender calls* in the worst case is $O(|I||U|)$.

5.5. ExpRebuild Search

The **ExpRebuild** search approach extends *GreedyGrow* by incorporating *items explanatory power* (Eq. 4) to guide item selection. Specifically, exploiting explanatory power, it prioritizes items whose removal is expected to most strongly

Algorithm 5. ExpRebuild (I, t)

Input: I: interacted items; t: target item;
Output: CF: counterfactual explanation;
Parameters: π: group recommender;
$f_1, f_2, \ldots, f_d$: item metrics functions
Variables: H: items list; i: item; S: selected items set;
inflScore: influence score ; *cfFound*: boolean variable; P: selected items from S

1 $H \leftarrow \emptyset$
2 **foreach** $i \in I$ **do** //compute items scores & initialize items list
3 $i.score \leftarrow \sum_{g=1}^{d} f_g(i)$ //compute item score by aggregating item metrics
4 insert i into H
5 sort H in descending order based on items scores
6 $S \leftarrow \emptyset$
7 **for** $j \leftarrow 1$ **to** $|H|$ **do** //select top items
8 $S \leftarrow S \cup H[j]$ //items set to be checked as counterfactual
9 *cfFound*, *inflScore* $\leftarrow$ checkForCFandComputeExplPwr(I, t, S) //check if S is counterfactual and compute $H[j]$ influence score
10 $H[j].pow \leftarrow inflScore/|S|$ //set explanatory power score to item $H[j]$
11 **if** *cfFound* = true **then break**
12 **if** *cfFound* = true **then** //rebuild based on explanatory power
13 sort S in descending order based on the explanatory power score
14 $P \leftarrow \emptyset$
15 **for** $j \leftarrow 1$ **to** $|S|$ **do**
16 $P \leftarrow P \cup S[j]$ //items set to be checked as counterfactual
17 *cfFound* $\leftarrow$ isCF(I, P, t)
18 **if** *cfFound* = true **then return** P
19 **return** S;
20 **else** //counterfactual not exist
21 **return** $\emptyset$

degrade the rank of the target item, or eliminate it entirely from the recommendation list.

Initially the method applies the *GreedyGrow* procedure to efficiently find an initial counterfactual explanation. During this process, the algorithm computes the explanatory power (Equation 4) of each item.

After an initial explanation is identified, the items are sorted in descending order of explanatory power, placing the most impactful items first. The algorithm then constructs a new explanation incrementally: starting from an empty set, items are added one by one in order of decreasing explanatory power. After each addition, the recommender is invoked to check whether the growing set constitutes a counterfactual explanation.

This approach offers a compromise between minimality and interpretability, aiming to produce more compact explanations grounded in a transparent attribution of item effects.

Procedure 2: checkForCFandComputeExplPwr(I, t, S)

Input: I: interacted items; t: target item; S: interacted items to be checked as counterfactual;
Output: $isCF$: boolean value (if S is a counterfactual); $explPower$: explanatory power score of item set S
Parameters: π: group recommender
Variables: L: recommendation list generated considering $I \setminus S$ as interacted

1 $L \leftarrow \pi(I \setminus S)$ //call recommender system
2 **if** $t \notin L$ **then**
3 $\quad isCF \leftarrow$ true
4 $\quad explPower \leftarrow 1$
5 **else**
6 $\quad isCF \leftarrow$ false
7 $\quad explPower \leftarrow \frac{\mathsf{rank}(t,L)-1}{|L|}$ //compute S explanatory power (Eq. 4)
8 **return** $isCF$, $explPower$

Algorithm Description. Algorithm 5 implements the *ExpRebuild* strategy. The algorithm begins by computing the total score for all items and sorting them into a ranked list H (*lines* 1–5). It then incrementally builds a candidate set S (*lines* 6–11), invoking the procedure checkForCFandComputeExplPwr (Proc. 2) at each step to determine whether a counterfactual explanation has been found and to compute the explanatory power of the current set.

Once a counterfactual explanation is identified, the items in S are ordered according to their explanatory power (*line* 14). The algorithm then reconstructs the explanation by incrementally adding items in descending explanatory power order (*lines* 15–18), checking after each addition whether the counterfactual condition holds.

The first subset that satisfies the counterfactual condition is returned as the final explanation. If no such subset is found, the full set S is returned (*line* 19). If no counterfactual explanation is identified during the initial phase, the algorithm returns the empty set (*line* 21).

Complexity: Time, Space, and Recommender Calls.
Time Complexity: In *line* 3 item scores are computed for all items $i \in I$ using d metric functions $f_1, \ldots, f_d$. As shown in Section 5.1, the complexity to compute the metric functions for an item is $O(|U|\phi)$. Thus, computing all item scores has a total cost: $O(|I||U|\phi)$.

Sorting the scored list H costs $O(|I| \log |I|)$. The main loop (*lines* 7–11) adds items one-by-one to the candidate set S and invokes the checkForCFandComputeExplPwr procedure at each step. In the worst case, the loop is performed $|I|$ times.

The checkForCFandComputeExplPwr procedure calls the recommender system once (*line* 1), and computes the item explanatory power score (Eq. 4), which can be done in $O(1)$ (if a hash-based indexing is used over L). Hence, each invocation of checkForCFandComputeExplPwr costs: $O(\phi)$.

Therefore, the total cost of the main loop (*lines* 7–11) is: $O(|I|\phi)$.

Next, if a counterfactual explanation is found (*line* 12), the algorithm sorts S based on item explanatory power scores, requiring $O(|I| \log |I|)$. Then, in the loop (*lines* 15–18), the algorithm attempts to rebuild a more concise valid explanation set $P \subseteq S$ by adding one item at a time and calling isCF. In the worst case, $|S| = |I|$, so this phase invokes the recommender system at most $|I|$ times, resulting in $O(|I|\phi)$.

Summing up all parts, the overall *worst-case time complexity* of the *ExpRebuild* algorithm is:

$$O\big(|I||U|\phi + |I| \log |I| + |I|\phi + |I| \log |I| + |I|\phi\big) = O\left(|I||U|\phi + |I| \log |I|\right)$$

Space Complexity: The *space complexity* is $O(|I|)$, for storing lists H, S, and P.

Recommender Calls: In the worst case, the number of recommender calls for metrics computations is $O(|I||U|)$, during the grow phase (via checkForCFandComputeExplPwr) is $O(|I|)$, and $O(|I|)$ calls during the rebuild phase (via isCF). Therefore, in the worst case, the *total number of recommender calls* is $O(|I||U|)$.

6. Experimental Evaluation

We evaluate our methods on MovieLens 10M and Amazon, examining explanation size, recommender-call cost, interpretability, and fairness across group sizes. We evaluate the impact of Pareto-filtering and we also compare our methods with an exhaustive search approach. Finally, we assess the stability of the results through bootstrap resampling and analyze utility under different weights for minimality and interpretability.

Table 2 summarizes the main findings of the experimental evaluation across five key characteristics of the proposed methods: cost, explanation size, fairness, interpretability, and utility. These findings are further discussed in Section 6.9.

6.1. Experimental Setting

Datasets. We employed two benchmark datasets, *MovieLens* 10M [15] and *Amazon* [17], which differ considerably in their characteristics. While MovieLens represents a dense and well-studied domain of movie ratings, the Amazon dataset is much sparser and reflects a more heterogeneous item space. Table 3 presents the characteristics of each dataset.

Evaluated Methods. We evaluate each of the proposed methods in two variants: with and without Pareto filtering.

Table 2
Summary of the main experimental findings and trade-offs among the proposed methods.

Method	Cost	Size	Fairness	Interpretability	Utility
GreedyGrow Sec. 5.3	Very Low	Very Large	Very Low	High	Lower
ExpRebuild Sec. 5.5	Low	Moderate	Low	Very High[a]	Highest on MovieLens at $\alpha = 0.3, 0.5$
Grow&Prune Sec. 5.4	Moderate	Small	High	Moderate	Highest on MovieLens at $\alpha = 0.7$
FixedWindow Sec. 5.2	High	Very Small	Very High	Low	Highest on Amazon
Pareto-filtering* Sec. 5.1	Mixed[b]	Mixed	Mixed	Mixed	Mixed[c]

* Pareto filtering is used as an optional preprocessing step of each algorithm.

[a] *ExpRebuild* leads in interpretability on MovieLens; *GreedyGrow* and *Grow&Prune* have similar scores on Amazon.

[b] Filtering increases calls for all methods on MovieLens Group 5, but reduces them for *GreedyGrow*, *Grow&Prune*, and *ExpRebuild* in Group 10 on both datasets.

[c] Filtering lowers utility on MovieLens but increases it on Amazon.

Table 3
Number of Users, Items, and Ratings

Dataset	Users	Items	Ratings
MovieLens	69,878	10,681	10,000,054
Amazon	344,747	373,665	5,573,065

The *Pareto-filtering* variants apply the Pareto algorithm as a preprocessing step to reduce the candidate set. The variant without filtering are denoted as *Sorted List*.

We also compare our methods with an exhaustive search (*Exhaustive*), which examines item combinations in increasing order of cardinality until a valid counterfactual explanation is identified. We implemented all methods in Python 3.12.7, using NumPy, pandas, SciPy, and scikit-learn for numerical computations, data processing, and sparse matrix operations.

Group Formation. To represent realistic group recommendation scenarios, we simulated groups of two sizes: five members (*Group 5*), reflecting smaller and more common decision-making units, and ten members (*Group 10*), representing larger and more diverse groups. We limited our analysis to users with a minimum of 50 ratings to ensure sufficient preference information and minimize noise in the recommendation process. This choice also increases the size of the candidate item pool, creating a more challenging environment for identifying group counterfactual explanations. We randomly generate 50 groups of each size.

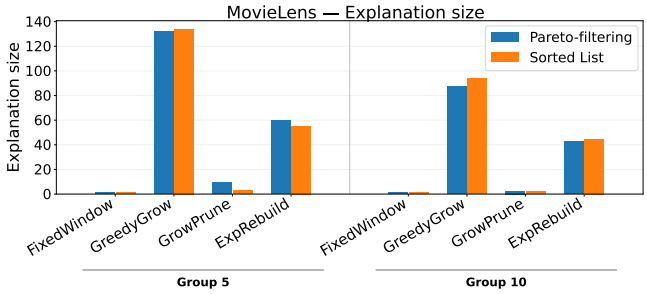
Furthermore, regarding the experiments where exhaustive search is used, additional groups are constructed. Due to the high computational complexity of exhaustive search, evaluating it on arbitrary groups is impractical within a reasonable time. To enable the comparisons with our methods, we selected 50 randomly generated groups of five users for which exhaustive search was able to identify a valid counterfactual explanation within a budget of 1,000 recommender calls. We refer to this selected set as *Group 5-Exst*. Although this selection focuses on groups where Exhaustive method succeeds within the call budget, it enables a direct comparison with our methods.

To assess the sensitivity of the results to the selected groups, we additionally perform a bootstrap stability analysis.

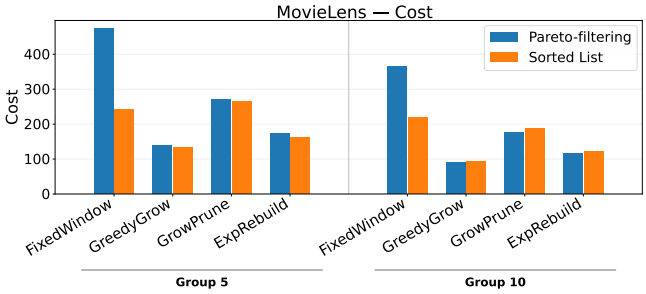
Evaluation Scenario. We report results averaged over the 50 randomly generated groups for each group size. Each group recommendation list contains 10 items. To ensure consistency and reduce randomness, we select the top-ranked item in the original group recommendation list as the target item. An explanation is considered successful if removing the items in the explanation causes the target to disappear from the top-10 recommendation list. We use a budget of 1,000 recommender calls and set the initial window size to $w = 15$ for the *FixedWindow* algorithm.

Evaluated Metrics. We evaluate the methods in terms of (a) explanation size; (b) cost (number of recommender calls); (c) interpretability; (d) fairness; and (e) utility. We further assess the stability of the results using bootstrap confidence intervals.

Recommender System. Group recommendation lists were generated through a two-step process. First, individual recommendation scores were predicted for each group member using user-based collaborative filtering with cosine similarity. The recommender considered all other users without restricting the neighborhood to a fixed number of users. It computed predictions using similarity-weighted ratings, assigning a score of zero when no similarity information was available. The predicted scores were normalized to a

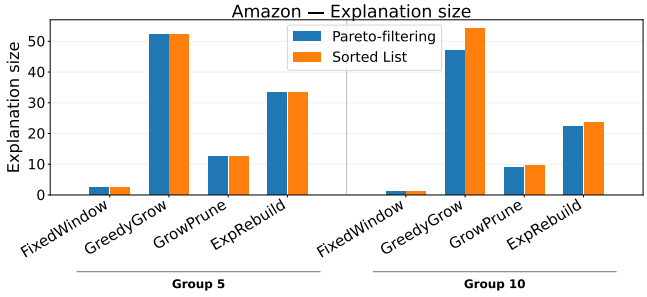


(a) Explanation size

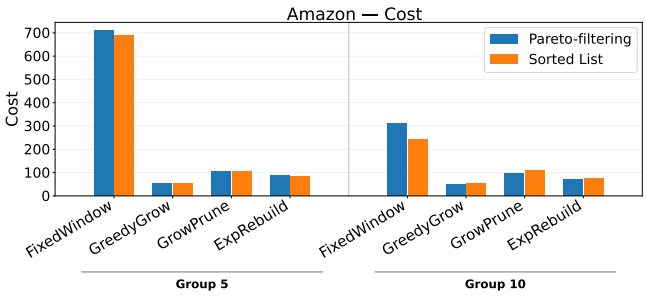


(b) Cost

Figure 2: Explanation size and Cost for MovieLens (Group 5 & Group 10).



(a) Explanation size



(b) Cost

Figure 3: Explanation size and Cost for Amazon dataset (Group 5 & Group 10).

common range of $[0, 5]$, and items already rated by each user were excluded from their individual recommendations.

Second, group-level scores were computed by averaging the predicted scores across all group members. Items previously rated by any group member were excluded from the final candidate set, and the 10 highest-scoring items were selected to form the group recommendation list.

For counterfactual evaluation, we removed the corresponding items from group members' rating profiles and recomputed their individual predictions and group recommendations. This configuration provides a consistent recommendation setting for evaluating our counterfactual explanation methods.

6.2. Explanation Size and Cost

MovieLens. Figure 2 reports the explanation size and cost for the MovieLens dataset, comparing groups of sizes 5 and 10 with and without Pareto-filtering.

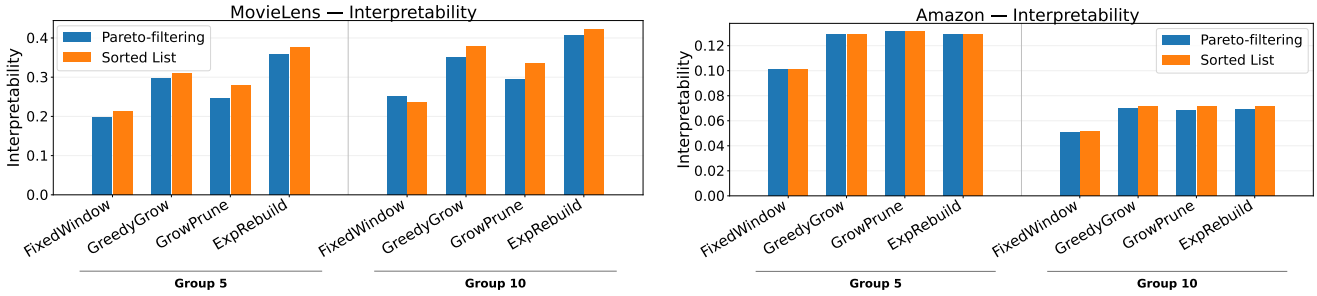
For groups of 5, GreedyGrow requires the fewest calls in both variants but produces the largest explanations. With Sorted List, it requires 134 calls on average, compared with 265 for Grow&Prune and 243 for FixedWindow. However, its explanations contain 134 items on average, compared with 2.9 for Grow&Prune and 1.3 for FixedWindow. ExpRebuild provides an intermediate alternative, requiring approximately 160 calls and producing explanations of 55 items. With Pareto-filtering, Grow&Prune explanation size increases from 2.9 to 9.3 items (see also Sect. 6.3).

For groups of 10, Grow&Prune and FixedWindow again produce compact explanations, while GreedyGrow remains the least expensive. In particular, Grow&Prune returns 2.41 items with Sorted List and 2.48 with Pareto-filtering, at a substantially lower call cost than FixedWindow.

Amazon. Figure 3 shows explanation size and cost for Amazon. For groups of 5, GreedyGrow requires the fewest calls (approximately 53) but produces the largest explanations (about 52 items). With Sorted List, Grow&Prune reduces the average explanation size to about 12 items, at a cost of 105 calls. ExpRebuild offers an intermediate alternative, producing 33 items with 84 calls. FixedWindow returns the shortest explanations (2.4 items), but at a much higher cost: approximately 688 calls with Sorted List and 709 with Pareto-filtering. Its local refinement becomes expensive when a suitable counterfactual window is difficult to locate.

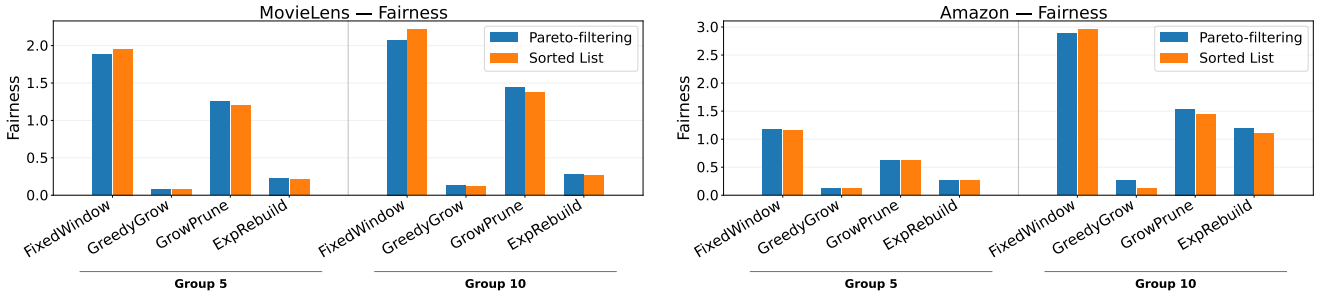
For groups of 10, the same trade-off is observed. GreedyGrow requires the fewest calls, while FixedWindow produces the shortest explanations but has the highest cost. Grow&Prune and ExpRebuild offer intermediate alternatives.

Summary. Overall, GreedyGrow requires the fewest recommender calls but produces the largest explanations, whereas FixedWindow typically returns the shortest explanations at a substantially higher cost. Grow&Prune offers a favorable compromise between explanation size and cost, while ExpRebuild provides an intermediate alternative.



(a) MovieLens (b) Amazon

Figure 4: Explanation Interpretability for MovieLens and Amazon (Group 5 and 10).



(a) MovieLens (b) Amazon

Figure 5: Explanation fairness for MovieLens and Amazon (Group 5 and 10).

6.3. Methods with Pareto-filtering

Figures 2 and 3 compare the Sorted List and Pareto-filtering variants for both group sizes. Pareto-filtering does not provide a uniform reduction in call cost. For MovieLens (Fig. 2) in Group 5, it increases the cost of all four heuristics and raises the mean explanation size of Grow&Prune. For Group 10, it reduces the call cost of GreedyGrow, Grow&Prune, and ExpRebuild, while their explanation sizes change less markedly.

For Amazon (Fig. 3) in Group 5, the two variants produce similar explanation sizes and call costs. For Group 10, Pareto-filtering reduces both measures for GreedyGrow, Grow&Prune, and ExpRebuild.

Summary. Overall, Pareto-filtering offers no consistent advantage across datasets and group sizes. It reduces cost for several algorithms in Group 10, particularly on Amazon, but may increase cost or explanation size in MovieLens Group 5.

6.4. Explanation Interpretability

We evaluate recognition-based interpretability using Eq. 8, which combines item recognition within the group and among users outside it. Figures 4a and 4b show different patterns across the two datasets. On MovieLens (Fig. 4a), ExpRebuild obtains the highest score in both group-size configurations and variants, with a score around 0.36–0.42. GreedyGrow also returns relatively recognizable items, while Grow&Prune and FixedWindow return shorter explanations with lower recognition scores. This indicates that compact explanations are not necessarily the most recognizable to group members.

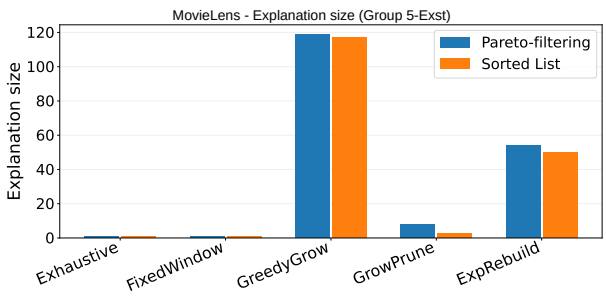
On Amazon (Fig. 4b), interpretability scores are considerably lower, consistent with the dataset's sparser interactions. For groups of size 5, GreedyGrow, Grow&Prune, and ExpRebuild achieve similar interpretability, around 0.13, while FixedWindow is closer to 0.10. For groups of size 10, the first three methods again obtain comparable scores, around 0.07. The difference between group sizes is more pronounced on Amazon than on MovieLens.

Pareto-filtering does not consistently improve interpretability. In some configurations, the Sorted List variant produces explanations containing more familiar items. ExpRebuild is particularly competitive on MovieLens, whereas the three grow-based methods perform similarly on Amazon.

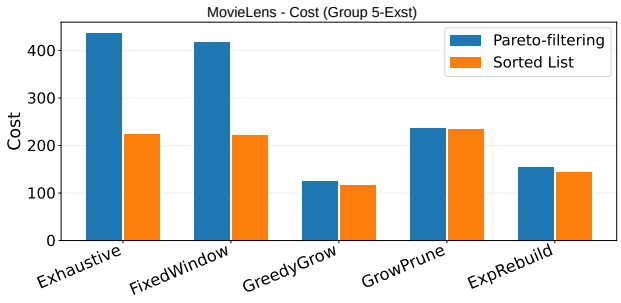
Summary. Overall, ExpRebuild achieves the highest interpretability on MovieLens, whereas GreedyGrow, Grow&Prune, and ExpRebuild obtain similar scores on Amazon. Pareto-filtering does not consistently improve interpretability.

6.5. Explanation Fairness Analysis

Figure 5 presents the fairness scores (Eq. 11) for the MovieLens and Amazon datasets. For MovieLens, FixedWindow obtains the highest scores, followed by Grow&Prune. For groups of 5 members, their scores are approximately 1.9 and 1.2, respectively, compared with less than 0.24 for ExpRebuild and 0.08 for GreedyGrow. The same ordering holds for groups of 10 members. A similar pattern is observed for Amazon. FixedWindow and Grow&Prune obtain higher fairness scores than ExpRebuild and GreedyGrow. In particular, for groups of size 10,

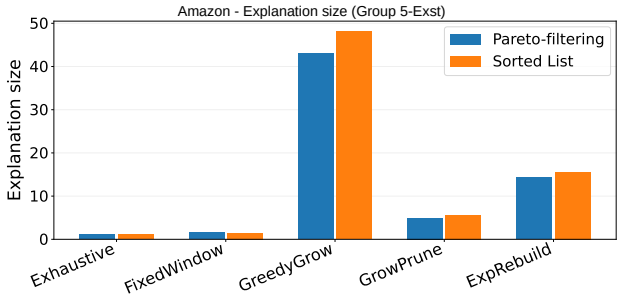


(a) Explanation size

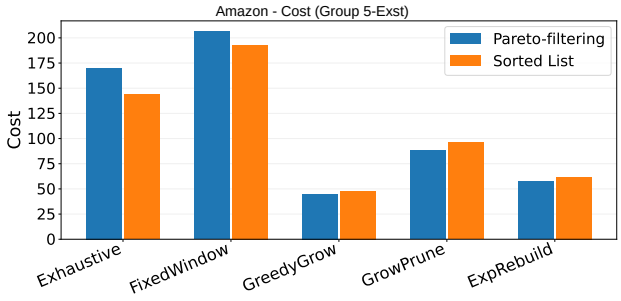


(b) Cost

Figure 6: Explanation size and Cost on MovieLens for the Exhaustive Search setting (Group 5-Exst).



(a) Explanation size



(b) Cost

Figure 7: Explanation size and Cost on Amazon for the Exhaustive Search setting (Group 5-Exst).

FixedWindow achieves fairness scores of approximately 2.9, while Grow&Prune obtains approximately 1.5.

Since the fairness metric is based on the distribution of absolute item counts among group members, fairness scores should be interpreted alongside explanation size.

Summary. Overall, FixedWindow achieves the highest fairness scores, followed by Grow&Prune, across both datasets and group sizes.

6.6. Comparison with Exhaustive Search

We compare the proposed algorithms with the Exhaustive approach on the groups of 5 members (Group 5-Exst) defined in the experimental setting.

Figure 6 presents the MovieLens results. With Sorted List, Exhaustive returns explanations of approximately 1.07 items at 223 calls. FixedWindow returns 1.14 items at a similar cost of 221 calls. Grow&Prune returns approximately 2.6 items at 234 calls. GreedyGrow and ExpRebuild use approximately 117 and 144 calls, respectively, while returning larger explanations. The comparison illustrates the different strengths of the proposed methods: FixedWindow approaches the compactness of Exhaustive, whereas GreedyGrow and ExpRebuild reduce calls at the expense of explanation size.

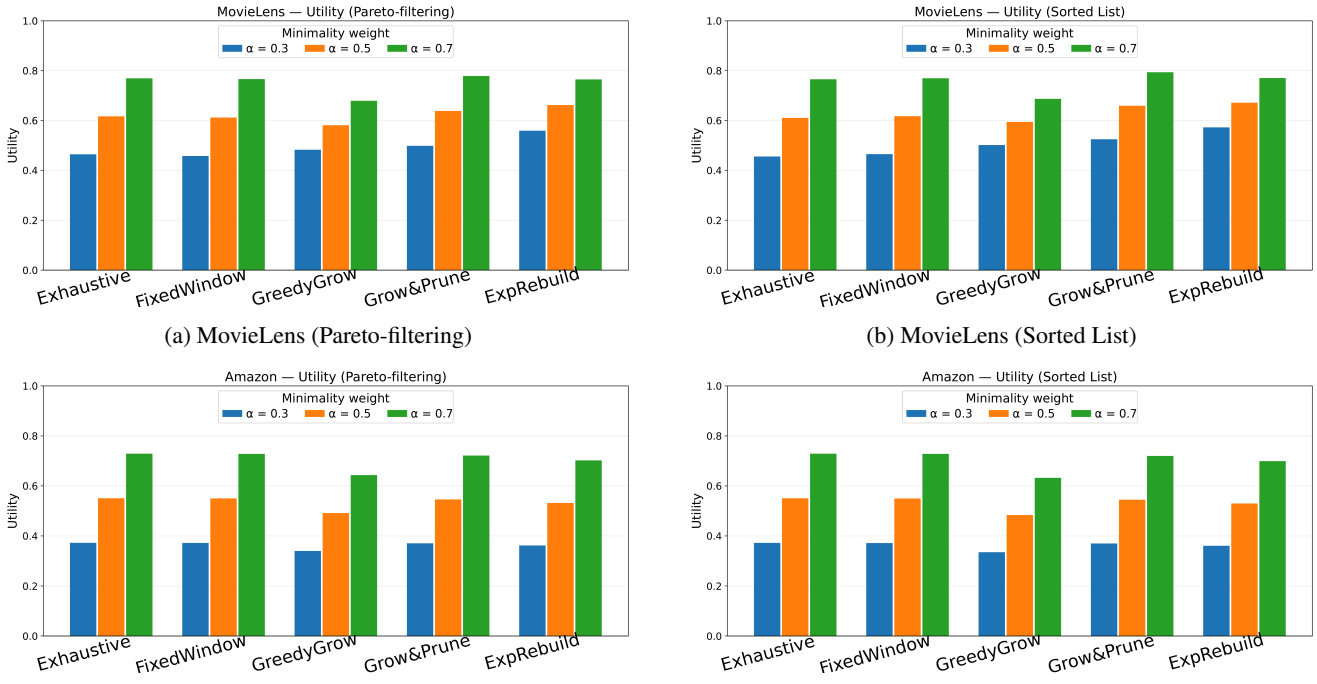
For Amazon (Fig. 7), Exhaustive returns approximately 1.13 items at 144 calls with Sorted List. FixedWindow produces similarly short explanations (1.5 items), although at a higher cost. Grow&Prune reduces the mean size from 48.1 items for GreedyGrow to 5.6 items, using approximately 96 calls. ExpRebuild returns 15.6 items at about 62 calls, while GreedyGrow requires 48 calls. In general, on Amazon, these three heuristics use fewer calls than Exhaustive, while FixedWindow uses more.

Summary. Grow&Prune is particularly effective when compactness must be balanced against computational cost. FixedWindow returns small explanations in both datasets, with average sizes close to those of Exhaustive. This is consistent with its local exhaustive refinement, which searches for smaller counterfactual subsets within a selected window.

6.7. Explanation Utility

To evaluate the utility of the generated explanations (Eq. 10), we employ a weighted sum of minimality and interpretability, defined as $\alpha \min(E) + (1 - \alpha)\,\mathrm{interpr}(E)$, where $\alpha \in [0, 1]$. The weight α determines the relative importance of the two metrics. We consider three configurations, $\alpha = 0.3, 0.5, 0.7$, and also report results for the Exhaustive.

Figure 9 shows that the preferred method depends on the dataset and the weight. On MovieLens, ExpRebuild has the highest mean utility at weights 0.3 and 0.5, while Grow&Prune leads at 0.7. This ordering holds for both Sorted List and Pareto-filtering. FixedWindow returns shorter explanations, but does not obtain the highest utility at any of the tested weights on MovieLens. On Amazon, FixedWindow is the best-performing heuristic and remains very close to Exhaustive, with mean utility differences below 0.001; Grow&Prune follows.



(a) MovieLens (Pareto-filtering)

(b) MovieLens (Sorted List)

(c) Amazon (Pareto-filtering)

(d) Amazon (Sorted List)

Figure 8: Explanation utility for MovieLens and Amazon under varying weights for minimality and interpretability (Group 5-Exst).

Figure 8 presents the results for groups of size 10. The algorithms show similar trends across utility configurations, suggesting that group size does not substantially affect performance.

Summary. Overall, no heuristic dominates utility across both datasets and all tested weights. The results show the strengths of each proposed method: ExpRebuild achieves higher utility when interpretability is emphasized, particularly on MovieLens, whereas Grow&Prune and FixedWindow are more competitive when minimality is given greater weight.

6.8. Stability Analysis

We assess the sensitivity of the MovieLens results for groups of five members in the exhaustive search setting (Group 5-Exst). For each algorithm and search variant, we resample groups with replacement and recompute the mean over 1,000 bootstrap repetitions, and report the corresponding 95% confidence intervals. The analysis concerns variation under group resampling rather than independent executions of the algorithms. Figure 10 shows the uncertainty in (a) cost, (b) explanation size, and (c) interpretability under resampling of the evaluated groups.

GreedyGrow has the lowest mean cost and ExpRebuild the highest mean interpretability in both search variants. With Sorted List, GreedyGrow requires approximately 117 calls, with a 95% interval of 93–143, compared with 145 calls and an interval of 119–175 for ExpRebuild. Their recognition scores are approximately 0.363 and 0.424, with intervals of 0.344–0.384 and 0.402–0.445, respectively.

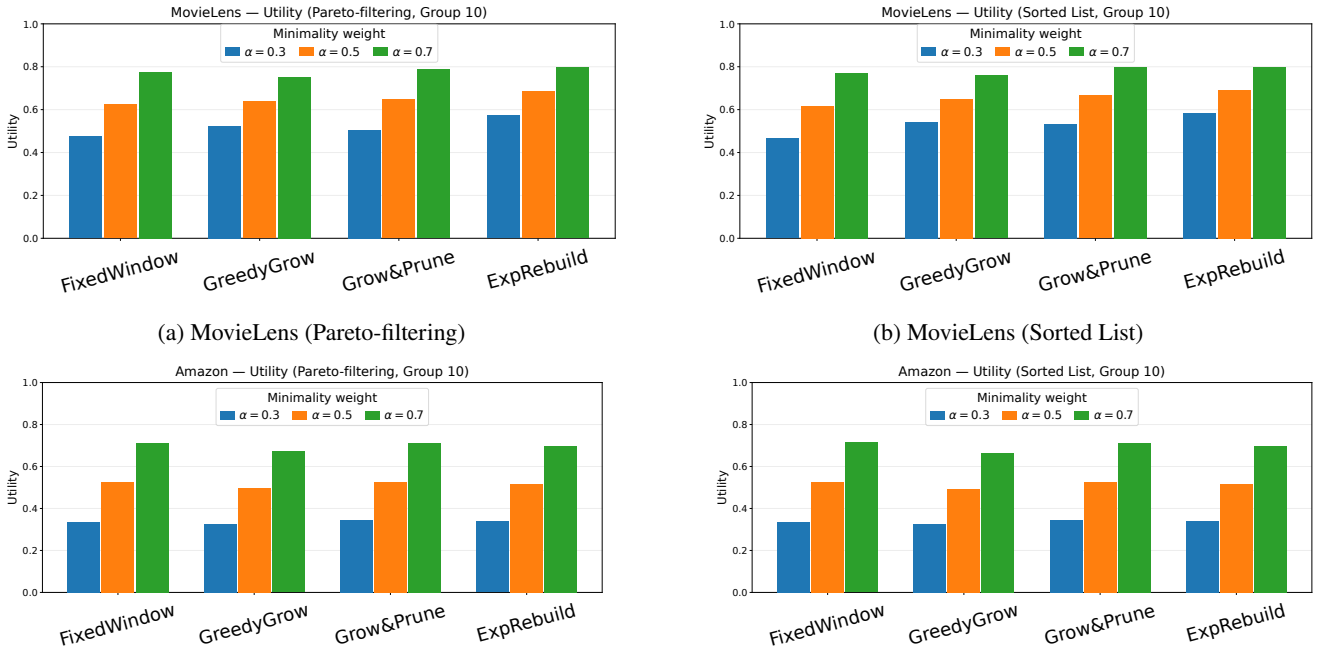
FixedWindow returns short explanations with Sorted List: approximately 1.14 items, with an interval of 1.05–1.25, compared with 1.06 items and 1.02–1.13 for Exhaustive. Grow&Prune has a mean size of 2.58 items and an interval of 2.15–3.08 with Sorted List. Its Pareto-filtering interval is much wider, 2.11–19.46, indicating greater uncertainty in mean explanation size under group resampling.

Summary. Overall, bootstrap resampling preserves the main relative patterns: GreedyGrow has the lowest call cost, ExpRebuild the highest interpretability, and FixedWindow produces short explanations. However, the wider explanation-size interval for Grow&Prune with Pareto-filtering indicates greater sensitivity to the sampled groups in that configuration.

6.9. Results Discussion

The evaluation shows that the choice of algorithm depends on the relative importance of explanation size, cost, and item recognition. Table 2 summarizes the main findings. GreedyGrow uses the fewest calls in the evaluated configurations, while FixedWindow returns the shortest explanations among the proposed methods, often at substantially higher call cost. Grow&Prune reduces explanation size relative to GreedyGrow through additional refinement, whereas ExpRebuild provides the highest interpretability on MovieLens; on Amazon, the three grow-based methods have similar recognition scores. *Pareto filtering* offers no uniform improvement and increases FixedWindow cost in every main configuration.

The *exhaustive search* comparison also shows that fewer calls are not an advantage shared by every algorithm:



(a) MovieLens (Pareto-filtering)

(b) MovieLens (Sorted List)

(c) Amazon (Pareto-filtering)

(d) Amazon (Sorted List)

Figure 9: Explanation utility for MovieLens and Amazon under varying weights for minimality and interpretability (Group 10).

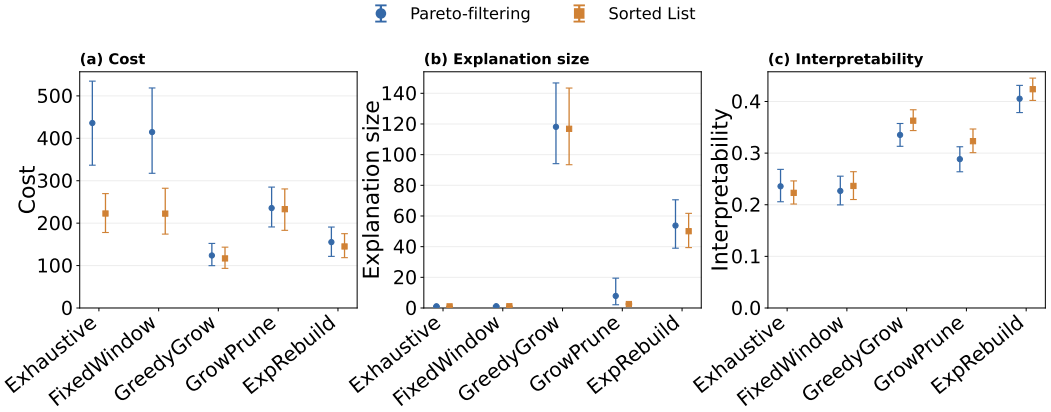


Figure 10: Bootstrap stability analysis on MovieLens for the Exhaustive Search setting (Group 5-Exst). Points show bootstrap estimates and error bars indicate 95% confidence intervals from 1,000 group-resampling repetitions for (a) cost, (b) explanation size, and (c) interpretability.

FixedWindow is close to Exhaustive on MovieLens but uses more calls on Amazon. In the *utility analysis* for groups of 5 members, ExpRebuild leads on MovieLens at weights 0.3 and 0.5 and Grow&Prune at 0.7, while Exhaustive leads on Amazon with FixedWindow very close. The *bootstrap analysis* indicates that uncertainty also depends on the variant, particularly for Grow&Prune explanation size under Pareto filtering.

7. Related Work

Recommender systems are a cornerstone of modern information access, assisting users in navigating large collections of items in domains such as e-commerce, media streaming, and online services. Early approaches were dominated by collaborative filtering and content-based techniques [1], which later evolved into hybrid and deep learning models to improve both scalability and accuracy. Beyond predictive performance, recent research has increasingly emphasized additional dimensions such as fairness, diversity, and explainability [34, 27, 57]. We next review existing work on group recommendations and counterfactual explanations.

7.1. Group Recommendations

Group recommendation has been widely studied [29]. A common strategy is to apply a standard recommendation model to each group member and then aggregate the individual recommendation lists [2, 4, 33, 42].

Different approaches have been proposed for the aggregation stage. For instance, [56] considers the varying

influence of individual group members when merging recommendations, while [8] employs attention networks and neural collaborative filtering to learn aggregation strategies directly from data. Along the same lines, [55] combines attention mechanisms with bipartite graph embeddings to capture member influence, whereas [38] leverages social networks enriched with user preferences and social interactions to determine group-level recommendations.

Group members interactions have also been explicitly modeled. For example, [48] simulates consensus-building as a series of voting processes and introduces a stacked social self-attention network to learn the underlying voting dynamics. To handle large groups, [35] proposes dividing them into subgroups with shared interests, identifying candidate media-user pairs for each subgroup, and then aggregating the resulting collaborative filtering lists. Another strategy is presented in [21], which introduces a two-phase recommender: the first phase seeks to maximize satisfaction at the group level, and the second phase refines recommendations to better satisfy individual preferences.

Utility-based approaches have also been explored. In [53], each member is assigned a utility score based on the relevance of items, and the system balances these utilities to generate a final group list. Similarly, [37] defines user utility as the similarity between individual and group-level recommendations and considers sets of Pareto-optimal items when constructing the list. Finally, [20] introduces the notion of rank-sensitive balance, where not only the top recommendation but also each successive prefix of the list must reflect a fair balance of member interests.

7.2. Counterfactual Explanations

Counterfactual explanations have emerged as a powerful approach to enhance the transparency and reliability of recommender systems. They clarify why certain items are recommended by identifying minimal changes in the input data that would alter the outcome. This form of explanation is intuitive, actionable, and model-agnostic, making it particularly valuable in contexts where user trust and accountability are essential. While counterfactuals have primarily been studied in individual recommendation scenarios, their potential is especially relevant to group settings, where understanding how different members' preferences shape the joint outcome is critical for transparency and acceptance.

Recent research has explored counterfactual explanations across diverse recommendation tasks. For instance, CAVIAR [18] modifies image embeddings to reveal the influence of visual features on recommendations, while [52] introduces CFairER, a fairness-aware counterfactual framework addressing demographic biases such as age and gender. Graph-based methods have also been proposed, including GNNUERS [30], which explains fairness issues in GNN-based recommendation models. Beyond recommended items, [43] studies "why-not" questions in collaborative filtering, providing explanations for items that were absent from recommendations, an idea later extended to graph-based systems in [3]. Temporal dynamics have also been incorporated: the CETD method [16] generates counterfactual explanations for sequential recommendation by considering evolving user behavior.

Other works focus on general frameworks and model-agnostic solutions. A study in [44] proposes a framework for producing explainable counterfactual recommendations by showing how small changes lead to different results, while [54] presents a learning-based approach that adapts explanations to user interaction histories. MACER [51] leverages reinforcement learning to provide model-agnostic, item-based counterfactual explanations, particularly suitable in settings with multiple aggregation models, such as group recommendations. LXR [5] offers a post-hoc, self-supervised approach to identify critical user interactions for a recommendation, and [52] studies attribute-level counterfactuals in heterogeneous information networks, focusing on item exposure disparities and fairness.

Guo et al. [14] use a modified group influence function to estimate the effect of removing training interactions on an individual recommendation. Here, the term "group" refers to a set of training samples, rather than a group of users receiving a common recommendation. Its search also relies on model derivatives, whereas our method evaluates observable responses of the group recommender. Sequential counterfactuals consider a different intervention space: GECE [39] searches over edited individual interaction sequences using genetic operations, and TRACE [22] uses query-limited sequence editing for ranking constraints, including removal of an item from the top-k list. Although the latter target-exclusion criterion is related, these methods explain an individual's ordered sequence rather than a group's joint recommendation.

A recent line of research has begun to explicitly consider group counterfactuals in relation to fairness. For example, CounterFair [25] proposes a framework that generates group counterfactuals for bias detection, fairness-aware recourse, and subgroup identification. By balancing recourse across sensitive groups, CounterFair demonstrates how counterfactual reasoning can serve not only as an explanatory tool but also as a means to mitigate algorithmic bias. This direction complements our focus by highlighting the potential of counterfactuals at the group level, though its emphasis lies in bias detection and subgroup discovery rather than in explaining group recommendations.

Together, these contributions have laid important foundations for counterfactual reasoning in recommender systems. However, most of this research remains centered on individual recommendations. In contrast, group recommendations introduce additional complexity, as they must reconcile diverse user preferences and balance fairness across members. To the best of our knowledge, counterfactual explanations tailored to group settings have not been systematically explored. Addressing this gap, we investigate how counterfactuals can be adapted to group recommendations, providing explanations that foster both trust and satisfaction among all group members.

8. Conclusions

In this paper, we introduced the first systematic framework for generating counterfactual explanations in group recommender systems. By formalizing group-level counterfactuals, we highlighted how explanations can reveal the influence of individual interactions on collective outcomes, while also addressing important dimensions such as minimality, interpretability, and fairness. To cope with the inherent combinatorial complexity of explanation discovery, we proposed a family of heuristic algorithms, as well as a *Pareto-based filtering* strategy. Our experimental evaluation on the MovieLens and Amazon datasets demonstrated clear trade-offs between efficiency, explanation size, interpretability, and fairness. In particular, *GreedyGrow* offered low computational cost, while *Grow&Prune* and *FixedWindow* produced compact explanations, and *ExpRebuild* showed promising interpretability results. The benefits of Pareto-filtering varied across algorithms and experimental settings.

Despite these findings, our evaluation is limited to static groups, a single recommendation approach, and average aggregation. Furthermore, the proposed heuristics do not generally guarantee globally minimal explanations, and their performance may depend on the characteristics of the underlying dataset.

Looking ahead, several research directions remain open. A promising avenue is to extend the framework to dynamic and sequential group recommendations, where both group composition and preferences evolve over time.

References

[1] Adomavicius, G., Tuzhilin, A., 2005. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. IEEE Trans. Knowl. Data Eng. 17.

[2] Amer-Yahia, S., Roy, S.B., Chawlat, A., Das, G., Yu, C., 2009. Group recommendation: Semantics and efficiency. VLDB Endowment (PVLDB) .

[3] Attolou, H., Tzompanaki, K., Stefanidis, K., Kotzinos, D., 2024. Why-not explainable graph recommender, in: IEEE International Conference on Data Engineering (ICDE).

[4] Baltrunas, L., Makcinskas, T., Ricci, F., 2010. Group recommendations with rank aggregation and collaborative filtering, in: ACM International Conference on Recommender Systems (RecSys).

[5] Barkan, O., Bogina, V., Gurevitch, L., Asher, Y., Koenigstein, N., 2024. A counterfactual framework for learning and evaluating explanations for recommender systems, in: ACM Web Conference (WWW).

[6] Bartolini, I., Ciaccia, P., Patella, M., 2008. Efficient sort-based skyline evaluation. ACM Trans. Database Syst. 33.

[7] Biega, A.J., Gummadi, K.P., Weikum, G., 2018. Equity of attention: Amortizing individual fairness in rankings, in: ACM International Conference on Research and Development in Information Retrieval (SIGIR).

[8] Cao, D., He, X., Miao, L., An, Y., Yang, C., Hong, R., 2018. Attentive group recommendation, in: ACM International Conference on Research and Development in Information Retrieval (SIGIR).

[9] Celar, L., Byrne, R.M.J., 2023. How people reason with counterfactual and causal explanations for artificial intelligence decisions in familiar and unfamiliar domains. Memory & Cognition 51.

[10] Chomicki, J., Godfrey, P., Gryz, J., Liang, D., 2003. Skyline with presorting, in: IEEE International Conference on Data Engineering (ICDE).

[11] Dwork, C., Hardt, M., Pitassi, T., Reingold, O., Zemel, R.S., 2012. Fairness through awareness, in: Innovations in Theoretical Computer Science.

[12] Ghazimatin, A., Balalau, O., Roy, R.S., Weikum, G., 2020. PRINCE: provider-side interpretability with counterfactual explanations in recommender systems, in: ACM Intrnational Conference on Web Search and Data Mining (WSDM).

[13] Godfrey, P., Shipley, R., Gryz, J., 2007. Algorithms and analyses for maximal vector computation. Journal on Very Large Data Bases (VLDBJ) 16.

[14] Guo, Y., Cai, F., Pan, Z., Shao, T., Chen, H., Zhang, X., 2024. A counterfactual explanation method based on modified group influence function for recommendation. Complex & Intelligent Systems 10.

[15] Harper, F.M., Konstan, J.A., 2015. The movielens datasets: History and context. ACM Trans. Interact. Intell. Syst. 5.

[16] He, M., An, B., Wang, J., Wen, H., 2023. Cetd: Counterfactual explanations by considering temporal dependencies in sequential recommendation. Applied Sciences 13.

[17] He, R., McAuley, J., 2016. Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering, in: World Wide Web Conference (WWW).

[18] Jain, N., Sharma, V., Sinha, G., 2024. Counterfactual explanations for visual recommender systems, in: ACM Web Conference.

[19] Kaffes, V., Sacharidis, D., Giannopoulos, G., 2021. Model-agnostic counterfactual explanations of recommendations, in: ACM Intrnational Conference on User Modeling, Adaptation and Personalization (UMAP).

[20] Kaya, M., Bridge, D., Tintarev, N., 2020. Ensuring fairness in group recommendations by rank-sensitive balancing of relevance, in: ACM International Conference on Recommender Systems (RecSys).

[21] Kim, J.K., Kim, H.K., Oh, H.Y., Ryu, Y.U., 2010. A group recommendation system for online communities. International Journal of Information Management .

[22] Kim, U., Choi, S.M., Kim, G.W., Lee, S., 2026. TRACE: Targeted ranking-aware counterfactual explanation for sequential recommendation, in: ACM Conference on Recommender Systems.

[23] Kliegr, T., Bahník, S., Fürnkranz, J., 2021. A review of possible effects of cognitive biases on interpretation of rule-based machine learning models. Artif. Intell. 295.

[24] Kung, H.T., Luccio, F., Preparata, F.P., 1975. On finding the maxima of a set of vectors. Journal of the ACM 22.

[25] Kuratomi, A., Lee, Z., Tsaparas, P., Junior, G.D., Pitoura, E., Lindgren, T., Papapetrou, P., 2024. Counterfair: Group counterfactuals for bias detection, mitigation and subgroup identification, in: IEEE International Conference on Data Mining (ICDM).

[26] Kusner, M.J., Loftus, J.R., Russell, C., Silva, R., 2017. Counterfactual fairness, in: Advances in Neural Information Processing Systems 30 (NIPS).

[27] Lenzi, E., Stefanidis, K., 2025. ADAPT: fairness & diversity for sequential group recommendations. Inf. Syst. 133.

[28] Lundberg, S.M., Lee, S.I., 2017. A unified approach to interpreting model predictions, in: Advances in Neural Information Processing Systems 30 (NIPS).

[29] Masthoff, J., 2015. Group recommender systems: Aggregation, satisfaction and group attributes, in: Recommender Systems Handbook. Springer.

[30] Medda, G., Fabbri, F., Marras, M., Boratto, L., Fenu, G., 2024. Gnnuers: Fairness explanation in gnns for recommendation via counterfactual reasoning. ACM Transactions on Intelligent Systems and Technology .

[31] Miller, T., 2019. Explanation in artificial intelligence: Insights from the social sciences. Artificial Intelligence 267.

[32] Narayanan, M., Chen, E., He, J., Kim, B., Gershman, S., Doshi-Velez, F., 2018. How do humans understand explanations from machine learning systems? an evaluation of the human-interpretability of explanation. CoRR abs/1802.00682.

[33] Ntoutsi, E., Stefanidis, K., Nørvåg, K., Kriegel, H., 2012. Fast group recommendations by applying user clustering, in: Conceptual

Modeling International Conference (ER).
[34] Pitoura, E., Stefanidis, K., Koutrika, G., 2022. Fairness in rankings and recommendations: an overview. Journal on Very Large Data Bases (VLDBJ) 31.
[35] Qin, D., Zhou, X., Chen, L., Huang, G., Zhang, Y., 2018. Dynamic connection-based social group recommendation. IEEE Trans. Knowl. Data Eng. .
[36] Reiter, R., 1987. A theory of diagnosis from first principles. Artif. Intell. 32.
[37] Sacharidis, D., 2019. Top-n group recommendations with fairness, in: ACM Symposium On Applied Computing (SAC).
[38] Salehi-Abari, A., Boutilier, C., 2015. Preference-oriented social networks: Group recommendation and inference, in: ACM International Conference on Recommender Systems (RecSys).
[39] Scarcelli, D., Betello, F., Perelli, G., Silvestri, F., Tolomei, G., 2025. Demystifying sequential recommendations: Counterfactual explanations via genetic algorithms. arXiv:2404.02147 .
[40] Sokol, K., Flach, P.A., 2019. Counterfactual explanations of machine learning predictions: Opportunities and challenges for ai safety, in: Workshop on Artificial Intelligence Safety.
[41] Stratigi, M., Bikakis, N., Stefanidis, K., 2025. Counterfactual explanations for group recommendations, in: International Workshop on Design, Optimization, Languages and Analytical Processing of Big Data (DOLAP).
[42] Stratigi, M., Pitoura, E., Nummenmaa, J., Stefanidis, K., 2022. Sequential group recommendations based on satisfaction and disagreement scores. J. Intell. Inf. Syst. 58, 227–254.
[43] Stratigi, M., Tzompanaki, K., Stefanidis, K., 2020. Why-not questions & explanations for collaborative filtering, in: Web Information Systems Engineering (WISE).
[44] Tan, J., Xu, S., Ge, Y., Li, Y., Chen, X., Zhang, Y., 2021. Counterfactual explainable recommendation, in: ACM International Conference on Information & Knowledge Management (CIKM).
[45] Tintarev, N., 2007. Explanations of recommendations, in: ACM International Conference on Recommender Systems (RecSys).
[46] Tintarev, N., Masthoff, J., 2007. A survey of explanations in recommender systems, in: IEEE International Conference on Data Engineering (ICDE).
[47] Vilone, G., Rizzo, L., Longo, L., 2020. A comparative analysis of rule-based, model-agnostic methods for explainable artificial intelligence, in: Irish Conference on Artificial Intelligence and Cognitive Science (AICS).
[48] Vinh Tran, L., Nguyen Pham, T.A., Tay, Y., Liu, Y., Cong, G., Li, X., 2019. Interact and decide: Medley of sub-attention networks for effective group recommendation, in: ACM International Conference on Research and Development in Information Retrieval (SIGIR).
[49] Wachter, S., Mittelstadt, B.D., Russell, C., 2018. Counterfactual explanations without opening the black box: Automated decisions and the GDPR. Harvard Journal of Law & Technology 31.
[50] Wang, D., Yang, Q., Abdul, A.M., Lim, B.Y., 2019. Designing theory-driven user-centric explainable AI, in: ACM International Conference on Human Factors in Computing Systems (CHI).
[51] Wang, Q., 2023. Reinforced model-agnostic counterfactual explanations for recommender systems, in: Proceedings of SPIE.
[52] Wang, X., Li, Q., Yu, D., Li, Q., Xu, G., 2024. Counterfactual explanation for fairness in recommendation. ACM Transactions on Information Systems 42.
[53] Xiao, L., Min, Z., Yongfeng, Z., Zhaoquan, G., Yiqun, L., Shaoping, M., 2017. Fairness-aware group recommendation with pareto-efficiency, in: ACM International Conference on Recommender Systems (RecSys).
[54] Yao, Y., Wang, C., Li, H., 2022. Learning to counterfactually explain recommendations. ArXiv abs/2211.09752.
[55] Yin, H., Wang, Q., Zheng, K., Li, Z., Yang, J., Zhou, X., 2019. Social influence-based group representation learning for group recommendation, in: IEEE International Conference on Data Engineering (ICDE).
[56] Yuan, Q., Cong, G., Lin, C.Y., 2014. Com: A generative model for group recommendation, in: ACM International Conference on Knowledge Discovery & Data Mining (KDD).
[57] Zhang, Y., Chen, X., 2020. Explainable recommendation: A survey and new perspectives. Found. Trends Inf. Retr. 14.